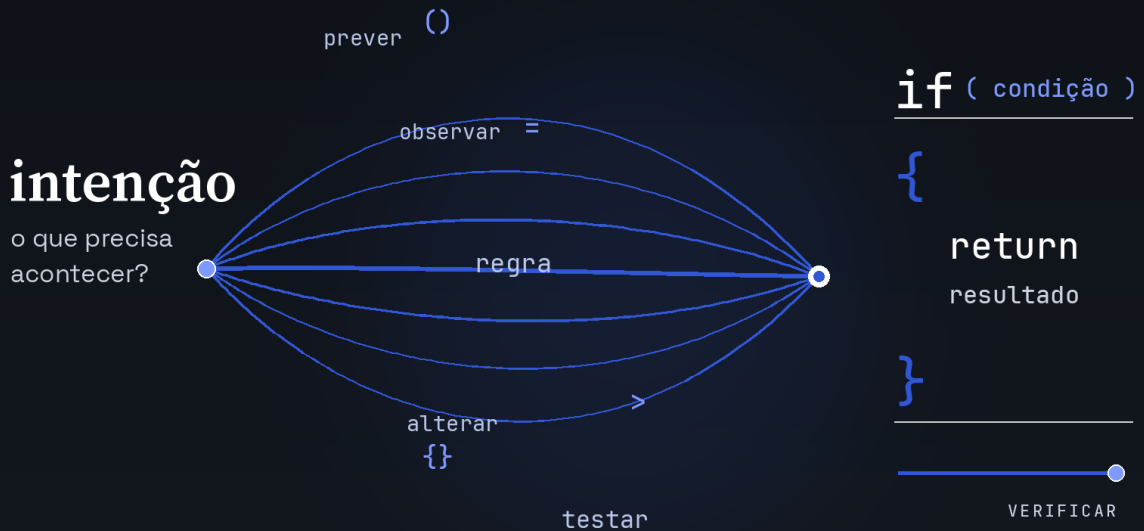


Programação na Era da IA



Direção, fundamentos e prática para aprender e
construir com IA sem perder o controle

Programação na Era da IA

**Direção, fundamentos e prática para aprender
e construir com IA sem perder o controle**

Marllon Panisset

MarllonCode 2026

Créditos

Programação na Era da IA *Direção, fundamentos e prática para aprender e construir com IA sem perder o controle*

Autor Marllon Panisset

Edição 1ª edição — 2026

Produção MarllonCode

Uso de inteligência artificial IA foi utilizada como ferramenta de apoio à pesquisa, redação, revisão, fact-check e produção editorial, sob direção, decisão e revisão do autor.

© 2026 Marllon Panisset. Todos os direitos reservados.

Este e-book é distribuído gratuitamente para uso pessoal. É permitido compartilhar somente o arquivo integral, sem alterações e sem cobrança, desde que os créditos e a autoria sejam preservados. Trechos podem ser citados com indicação do autor e da obra.

É proibida a revenda ou a incorporação comercial, total ou parcial, deste material sem autorização do autor.

Marcas, produtos e nomes de terceiros mencionados pertencem aos respectivos titulares. As referências são utilizadas para fins informativos, educacionais e de identificação.

Nota da edição

Esta é a edição de 2026 de *Programação na Era da IA*. Ferramentas, interfaces, produtos e serviços mencionados podem mudar depois da publicação. Por isso, o livro prioriza princípios, conceitos e critérios que continuem úteis além de uma versão específica.

Links e instruções técnicas foram verificados até **23 de agosto de 2026**. Se uma tela, um menu ou um procedimento estiver diferente, consulte a documentação oficial atual da ferramenta antes de continuar.

As atividades deste livro não certificam domínio de programação. Prever, explicar, alterar e testar são indicadores práticos de controle sobre o programa e o contexto trabalhados; formação mais ampla depende de novos ciclos, projetos e prática prolongada.

Apresentação

Este livro foi escrito para quem quer começar a programar, mas encontra linguagens, cursos, ferramentas, roadmaps e opiniões demais antes mesmo de conseguir decidir o próximo passo. Também foi pensado para quem já estudou alguma sintaxe, começou vários percursos e ainda não sabe separar o que precisa aprender agora do que pode deixar para depois.

A popularização da inteligência artificial acrescentou outra camada a essa confusão. Hoje é possível pedir uma explicação, gerar uma função ou iniciar um projeto em poucos segundos. Isso amplia o que uma pessoa iniciante consegue experimentar, mas não resolve automaticamente o problema da aprendizagem. Um programa funcionar não demonstra, por si só, que quem o produziu consegue explicar suas decisões, alterar uma regra ou recuperar o controle quando algo dá errado.

Programação na Era da IA propõe um percurso menor e mais concreto. Em vez de apresentar uma enciclopédia de tecnologias ou um roadmap que exige meses de preparação, o livro ajuda você a escolher prioridades, observar como regras se transformam em código e concluir uma pequena versão funcional. JavaScript aparece como linguagem de laboratório, não como uma decisão definitiva sobre sua carreira ou área de atuação.

O livro não promete domínio, emprego ou transformação profissional em poucas páginas. Também não pretende substituir um curso completo, documentação técnica ou prática prolongada. A proposta é mais específica: ajudar você a sair da confusão com critérios melhores para aprender, construir, verificar o que fez e escolher o próximo ciclo.

Para aproveitar o percurso, execute as atividades. Antes de rodar um exemplo, faça uma previsão. Quando o resultado aparecer, compare-o com o que esperava. Altere alguma coisa de forma intencional e teste novamente. Você pode consultar anotações, documentação e exemplos sempre que precisar; compreender não significa memorizar toda a sintaxe.

A IA pode participar desde o início, mas não é obrigatória. Use-a para receber explicações alternativas, confrontar hipóteses, organizar dúvidas

e investigar erros. Preserve para você as ações que deseja praticar: prever, definir o comportamento esperado, explicar decisões e verificar resultados. Se uma resposta parecer convincente, trate-a como algo que ainda precisa ser comparado com o código, a execução ou a documentação.

Você não precisa ler tudo de uma vez nem seguir o ritmo de outra pessoa. Precisa apenas realizar cada atividade com atenção suficiente para saber o que observou, o que conseguiu controlar e o que ainda permanece incompreensível. É assim que o livro deve ser usado: não como uma lista para terminar, mas como o primeiro ciclo que você consegue fechar.

Sumário

- 1 Seu próximo passo começa agora
- 2 Programar não é apenas escrever código
- 3 O que merece sua atenção agora
- 4 Aprender é fechar ciclos
- 5 IA como apoio, não como substituição
- 6 Reduza até caber
- 7 Construa em pequenas decisões
- 8 Funcionou. Mas você entendeu?
- 9 Escolha o próximo ciclo, não a vida inteira

Seu próximo passo começa agora

Você pode passar semanas tentando descobrir qual linguagem aprender, qual curso comprar, qual roadmap seguir e qual ferramenta de IA usar.

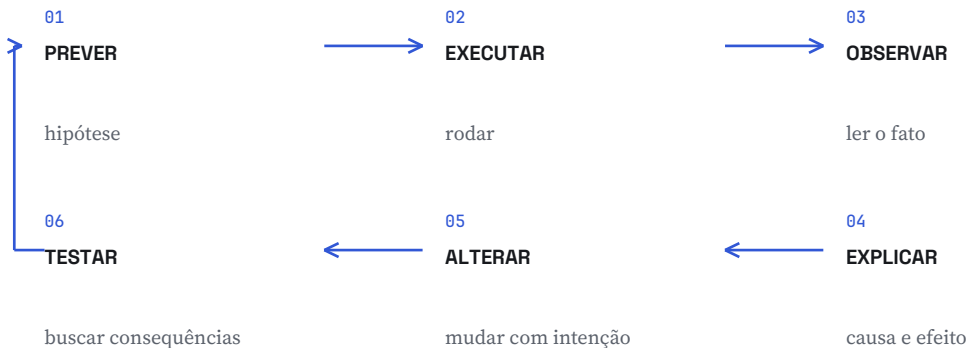
Enquanto isso, ainda não executou uma linha de código.

O problema, neste momento, não é falta de informação. É tentar tomar decisões demais antes de ter experiência suficiente para avaliá-las.

Você não precisa escolher sua trajetória inteira agora. Precisa concluir um ciclo pequeno:

PREVER → EXECUTAR → OBSERVAR → EXPLICAR → ALTERAR → TESTAR

DIAGRAMA D1



Depois desse ciclo, você continuará tendo dúvidas. A diferença é que já terá algo concreto para analisar.

Onde você está

Veja qual situação mais se aproxima da sua. Isso não é uma classificação permanente, e você pode se reconhecer em mais de uma.

Ainda não executei código

Você vê linguagens, símbolos e ferramentas, mas não sabe onde escrever uma instrução nem como fazer o computador executá-la.

Seu próximo passo é ter um primeiro contato sem transformar a preparação do ambiente em outra barreira.

Já acompanhei cursos ou tutoriais

Você reconhece parte da sintaxe e talvez já tenha copiado projetos. Ainda é difícil começar sozinho, explicar o que aconteceu ou decidir o que estudar em seguida.

Seu próximo passo é praticar um comportamento pequeno até conseguir prever, explicar e alterar o resultado.

Já usei IA para produzir código

Você descreve uma ideia e recebe código, mas não compreende todas as partes ou não sabe continuar quando algo falha.

Seu próximo passo não é abandonar a IA. É recuperar participação nas decisões e verificações.

Os três pontos de partida levam à mesma atividade: executar uma regra pequena e observável. O diagnóstico serve apenas para ajustar o próximo passo. Quem nunca executou código precisa de contato; quem já executou precisa de controle; quem gera com IA precisa de critérios para avaliar.

O que não precisa entrar neste primeiro ciclo

Você não precisa decidir agora qual será sua especialização, qual framework está em alta ou como construir uma aplicação completa. Também não precisa instalar várias ferramentas antes do primeiro resultado.

As quatro categorias que usaremos durante o livro ajudam a proteger sua atenção:

Prioridade	Neste primeiro ciclo
APRENDA AGORA	O mínimo necessário para executar e interpretar uma instrução
PRATIQUE AGORA	Prever, executar, observar, alterar e testar
ENTENDA QUE EXISTE	Linguagens, frameworks, bancos de dados e outras ferramentas
ESTUDE DEPOIS	O que não ajuda a compreender nem modificar o experimento atual

Colocar algo em **ESTUDE DEPOIS** não significa considerá-lo inútil. Significa apenas que ele não é pré-requisito para a próxima ação.

Seu primeiro contato com código

Usaremos JavaScript porque o navegador já consegue executá-lo. Isso não significa que desenvolvimento web será sua rota obrigatória nem que JavaScript é superior a outras linguagens. Aqui, ele funciona como linguagem de laboratório.

Abra uma nova aba no Google Chrome ou Microsoft Edge. Na barra de endereço, digite:

LITERAL TÉCNICO

```
about:blank
```

Pressione Enter e abra o console:

- no Windows ou Linux, pressione `Ctrl + Shift + J`;
- no macOS, pressione `Command + Option + J`.

Se aparecer um painel com várias abas, selecione **Console**. É ali que você digitará os comandos. Esses atalhos e a execução de expressões JavaScript no console estão documentados pelo Chrome.¹

Os comandos são curtos. Digite-os manualmente.

PREVER

Antes de escrever qualquer coisa, responda:

Qual resultado espero receber ao somar 2 e 2?

Registre a resposta, mesmo que pareça óbvia.

EXECUTAR E OBSERVAR

Digite no console:

JAVASCRIPT

```
2 + 2
```

Pressione Enter. O console deverá mostrar:

SAÍDA

```
4
```

Compare o resultado com sua previsão. Você forneceu dois valores, aplicou uma operação e recebeu uma saída observável.

Agora faremos algo menos óbvio.

Sem executar, preveja se a comparação abaixo produzirá `true` ou `false`:

JAVASCRIPT

```
25 < 20
```

As palavras `true` e `false` representam verdadeiro e falso.

Depois de registrar sua previsão, execute. O resultado será:

SAÍDA

```
false
```

Isso não é um erro. O símbolo `<` significa “menor que”. A instrução perguntou se 25 é menor que 20, e a resposta foi falsa.

ALTERAR

Troque apenas um valor:

JAVASCRIPT

```
10 < 20
```

Preveja antes de pressionar Enter. Depois execute. A saída será:

SAÍDA

```
true
```

Agora explique com suas palavras:

Por que `25 < 20` produz `false`, enquanto `10 < 20` produz `true`?

Não procure uma definição perfeita. Relacione a alteração ao resultado.

TESTAR O LIMITE

Sem executar, preveja:

JAVASCRIPT

```
20 < 20
```

Os valores são iguais, mas a pergunta continua sendo se o primeiro é menor que o segundo. Ao executar, o resultado será:

SAÍDA

```
false
```

Você verificou três situações:

Valor	Regra	Limite	Resultado
25	<	20	false
10	<	20	true
20	<	20	false

O último caso está exatamente no limite. Testar apenas números claramente menores ou maiores poderia esconder uma interpretação

incorreta da regra. Um valor igual ao limite força uma leitura precisa do símbolo.

O que você acabou de fazer

Os comandos são pequenos, mas o ciclo é real. Você definiu o que esperava, executou, observou, alterou e testou um caso diferente.

Cada ação responde a uma pergunta. A previsão registra sua interpretação; a execução mostra o que a instrução produz; a alteração relaciona mudança e comportamento; o teste do limite procura uma leitura apressada da regra.

Também encontrou três partes que aparecerão durante todo o livro:

- **entrada:** os valores comparados;
- **regra:** a comparação representada por <;
- **saída:** true ou false.

Por enquanto, basta reconhecer o mecanismo:

Valores entram, uma regra é aplicada e um resultado observável é produzido.

A primeira peça do Planejador

Ao longo do livro, desenvolveremos um pequeno programa chamado **Planejador da Próxima Sessão**.

A ideia é usar o tempo disponível e a existência de um bloqueio para recomendar uma ação. Uma das regras será:

- com menos de 20 minutos, exibir `Faça uma alteração pequena.`;
- com 20 minutos ou mais, exibir `Construa e teste uma melhoria.`

Você ainda não programou as mensagens nem o Planejador completo. Acabou de executar a parte mais básica da decisão:

O tempo disponível é menor que 20?

O programa crescerá somente quando uma nova parte contribuir para sua compreensão. Ele não precisa ganhar interface, banco de dados ou IA integrada para parecer mais importante.

Essa escolha também protege a transferência do aprendizado. Como o núcleo é uma comparação, você consegue enxergar a mesma decisão mesmo que, no futuro, ela apareça dentro de uma página, um aplicativo ou outra linguagem.

Onde a IA entra

A IA pode participar desde o começo, mas não precisa responder antes de você pensar.

Se tiver uma ferramenta disponível, tente perguntar:

Executei `25 < 20`, `10 < 20` e `20 < 20` em JavaScript. Não escreva outro código. Explique por que os resultados foram diferentes e depois faça uma pergunta para verificar se eu entendi.

USE IA PARA

receber outra explicação ou uma pergunta de verificação.

NÃO TERCEIRIZE

sua primeira previsão, a execução e sua explicação inicial.

VERIFIQUE

se a explicação corresponde ao que apareceu no console e se você consegue repeti-la sem copiar.

Sem IA, o percurso permanece completo. Preveja e execute `19 < 20` e `21 < 20` para conferir sua compreensão.

Explique os dois resultados como se estivesse respondendo a outra pessoa. Se a explicação depende da frase deste capítulo, leia novamente, feche o trecho e tente reconstruí-la. Consultar e tentar de novo faz parte da atividade.

Quando usar uma ferramenta, observe sua reação: comparou a resposta com o console ou a aceitou porque parecia bem escrita? Neste experimento, a execução é sua referência independente para avaliar a ajuda recebida.

Começar modificando não é compreender tudo

Minha relação com programação começou quando eu queria criar um servidor de Tibia para jogar com meus amigos.

Eu baixava servidores, configurava o `config.lua`, modificava arquivos XML e Lua e entrava no jogo para observar o resultado. Alterava mapas, quests, NPCs, monstros e magias. Quando algo dava errado, comparava arquivos, pesquisava em fóruns e tentava novamente.

Não considero que já programava naquela época. Eu conseguia localizar partes, modificar e testar, mas não compreendia bem programação e lógica. Mesmo assim, a exploração me ajudou a começar a enxergar regras por trás do jogo.

A limitação era não separar claramente três situações: funcionou; entendi por que funcionou; consigo modificar sem perder o controle. Curiosidade e tentativa abriram o caminho, mas algumas partes começavam a fazer sentido enquanto outras continuavam funcionando porque vinham prontas.

Este livro não substitui essa curiosidade por uma sequência rígida. Acrescenta critérios para você perceber quando está explorando, quando está compreendendo e quando precisa reduzir o problema para recuperar controle.

Sua próxima ação

Antes de seguir, confirme que consegue executar e explicar a comparação, alterá-la intencionalmente e citar pelo menos um assunto que pode esperar. Então complete:

Na próxima sessão, vou _____.

Escreva uma ação executável, como repetir as comparações com outros valores, explicar o que < faz ou seguir para o próximo capítulo e descrever o Planejador sem usar código. Evite respostas vagas como “estudar JavaScript”.

Se nunca tinha executado código, repetir a atividade com valores próprios é suficiente. Se já acompanhou cursos, justifique cada previsão.

Se costuma gerar código com IA, faça a previsão primeiro e use a ferramenta para confrontar sua explicação. A ação precisa dizer o que você fará quando sentar diante do computador.

Você ainda não escolheu sua carreira nem concluiu um curso de programação. Mas já saiu de “não sei por onde começar” para algo mais específico:

Executei uma regra, alterei seu comportamento, verifiquei o resultado e sei qual é minha próxima ação.

No próximo capítulo, vamos separar coisas que costumam ser tratadas como iguais: escrever código, gerar código, programar e desenvolver software.

Programar não é apenas escrever código

No capítulo anterior, você executou uma comparação, previu o resultado, alterou um valor e testou o limite.

Uma IA poderia ter digitado os mesmos símbolos:

JAVASCRIPT

```
25 < 20
```

O navegador devolveria o mesmo `false`. Para o computador, não importa quem digitou. Para quem está aprendendo, existe uma diferença.

Se você entendeu a pergunta representada pelo código, previu a resposta e soube alterar a comparação, participou do raciocínio. Se apenas copiou os símbolos e apertou Enter, conseguiu um resultado, mas ainda não assumiu controle sobre ele.

Código funcionando importa. Não é, sozinho, prova de compreensão.

O código representa uma decisão

A expressão `25 < 20` não surgiu do nada. Antes dela, existia uma pergunta:

Tenho menos de 20 minutos disponíveis?

Os números representam valores. O símbolo `<` representa uma comparação. O resultado informa se a afirmação é verdadeira ou falsa.

O código é uma maneira de expressar a pergunta em uma linguagem que o computador consegue executar. O trabalho começou antes dos símbolos, quando alguém decidiu:

- qual problema queria reduzir;
- qual informação seria utilizada;
- qual comportamento esperava;
- como verificaria a regra.

Podemos enxergar cinco camadas:

- 1 **Problema:** por que o programa deveria existir?
- 2 **Comportamento esperado:** o que deveria acontecer?
- 3 **Instruções:** quais passos e decisões produzem esse comportamento?
- 4 **Código:** como representar isso em uma linguagem?
- 5 **Execução e verificação:** o que realmente aconteceu?

Quando começamos diretamente pelo código, essas camadas ficam escondidas. Com IA, podemos receber dezenas de linhas antes mesmo de definir o que deveria acontecer.

As camadas também ajudam quando algo falha. Se o programa não executa, talvez exista um problema na representação. Se executa a regra contrária, código e comportamento esperado não estão alinhados. Se a saída parece correta, mas o problema permanece, talvez tenhamos implementado a coisa errada. Em vez de “meu projeto não funciona”, passamos a investigar uma camada específica.

Sintaxe e significado não são a mesma coisa

Compare:

JAVASCRIPT

```
25 <
```

JAVASCRIPT

```
25 > 20
```

A primeira expressão está incompleta. O computador não consegue interpretar sua forma: há um erro de sintaxe.

A segunda possui sintaxe válida e produz `true`. Mas, se nossa pergunta continua sendo “tenho menos de 20 minutos?”, ela representa a comparação contrária.

O primeiro código não executa. O segundo executa a regra errada.

Essa diferença continuará importante quando o programa crescer. Uma mensagem de erro pode indicar que a linguagem não conseguiu executar determinada forma. Já uma saída incorreta, sem erro visível, exige comparação com o comportamento esperado. “Rodou” e “resolveu a intenção” são verificações diferentes.

Há ainda uma terceira situação: o código executa e produz a saída esperada, mas você não consegue explicar, modificar ou verificar outros casos. Por isso, “executa?”, “representa a regra?” e “consigo controlá-lo?” são perguntas diferentes.

O que significa programar neste livro

Não precisamos fiscalizar quem é um “programador de verdade”. Para este percurso, usaremos uma definição prática:

Programar é transformar uma intenção em comportamento executável e verificável.

Isso inclui compreender um problema, definir entradas e resultados, organizar decisões, representá-las em código e comparar o real com o esperado. Escrever ou gerar código pode fazer parte do processo. A diferença está nas decisões que continuaram sob seu controle.

A questão não é proteger o ato de digitar. É preservar o raciocínio que permite avaliar o que foi digitado ou gerado.

Escrever, gerar, programar e desenvolver

Ação	O que pode demonstrar	O que não demonstra sozinha
Escrever código	Familiaridade com a linguagem e capacidade de representar instruções	Que o código resolve o problema correto
Gerar código	Capacidade de obter uma solução candidata com uma ferramenta	Que a solução está correta, foi compreendida ou pode ser mantida
Programar	Capacidade de transformar decisões em comportamento e verificá-lo	Que o resultado está pronto para uso por outras pessoas
Desenvolver software	Capacidade de lidar também com contexto, usuários, dados, falhas, manutenção e evolução	Que uma pessoa precisa dominar todas as tecnologias envolvidas

Na conversa cotidiana, “programar” e “desenvolver software” muitas vezes são usados como sinônimos. A distinção é útil porque um programa pequeno pode funcionar no seu computador sem enfrentar questões de dados, segurança, rede, manutenção e colaboração.

Quando outras pessoas passam a depender do resultado, surgem responsabilidades que o pequeno experimento ainda não possui: dados, falhas, manutenção e impacto das mudanças. Engenharia de software aprofunda práticas para lidar com essas restrições, mas não é pré-requisito para seu primeiro programa.

Por enquanto, evite duas confusões:

Um trecho de código não é automaticamente uma solução completa.

Uma solução que funcionou uma vez não está automaticamente compreendida, testada ou pronta para uso real.

Configurar, modificar e criar uma funcionalidade

Quando comecei a lidar com o site em PHP do meu OTServer, eu conseguia baixar o conjunto, configurar Apache e MySQL com XAMPP, conectar site e servidor e fazer pequenas mudanças de texto, imagem e aparência.

O que eu não conseguia era criar uma nova funcionalidade. Eu não compreendia a lógica em PHP nem o banco de dados o suficiente para decidir e implementar um comportamento novo.

Esse limite é mais preciso do que dizer apenas “eu sabia” ou “eu não sabia”. Eu controlava algumas partes do sistema e dependia de uma estrutura pronta em outras.

Algo parecido pode acontecer quando uma pessoa recebe um projeto de uma IA, troca um texto, uma cor ou um número e vê o resultado mudar. Isso pode ser uma boa entrada para explorar. Ainda não responde se ela entende onde está a decisão principal, quais valores influenciam o resultado e como investigar uma falha.

Você não precisa dominar uma linguagem antes de explorar. Ler, comparar e adaptar fazem parte do desenvolvimento. O avanço está em reconhecer a fronteira entre o que consegue explicar e alterar e o que ainda depende de uma estrutura pronta. Essa fronteira informa o próximo estudo e o tamanho seguro da próxima mudança.

Do problema ao algoritmo

Neste momento, pense em **algoritmo** apenas como uma sequência organizada de passos e decisões para chegar a um resultado.

O Planejador pode ser descrito assim:

- 1 receber a quantidade de minutos disponíveis;
- 2 comparar essa quantidade com 20;
- 3 se houver menos de 20 minutos, recomendar uma alteração pequena;
- 4 caso contrário, recomendar construir e testar uma melhoria;
- 5 mostrar a recomendação.

Ainda não escrevemos o programa completo, mas já temos um algoritmo informal.

Ele poderia ser representado em JavaScript, Python, Lua ou outra linguagem. A sintaxe mudaria; o problema e a decisão principal continuariam reconhecíveis.

Cada linguagem possui recursos, convenções e ambientes próprios. Ainda assim, reconhecer entrada, decisão, ordem e resultado evita que você recomece conceitualmente do zero ao encontrar outra sintaxe.

Comportamento esperado vem antes do código

Dizer que o programa “deve funcionar” é vago. Precisamos definir o que isso significa.

Minutos disponíveis	Comportamento esperado
10	Faça uma alteração pequena.
20	Construa e teste uma melhoria.
35	Construa e teste uma melhoria.

DIAGRAMA D2



A tabela torna visível uma decisão: o que acontece exatamente com 20 minutos?

Como a regra usa “menos de 20”, o valor 20 segue a segunda opção. Poderíamos definir “até 20”, mas então o comportamento esperado e o código precisariam mudar.

Prever não é tentar adivinhar o que o computador fará. É deduzir uma saída a partir da regra definida. Se você erra, a divergência informa que sua interpretação da regra, do valor ou do símbolo precisa ser revista.

Sem uma expectativa anterior, qualquer saída pode parecer aceitável depois que aparece. Com uma expectativa, você consegue comparar intenção e execução.

Antes de perguntar “qual símbolo uso?”, precisamos decidir “qual comportamento quero?”. A linguagem representa a decisão; não decide por nós.

ATIVIDADE

descreva antes de codificar

Não escreva código novo. Responda com frases curtas:

- 1 **Problema:** que dificuldade o Planejador tenta reduzir?
- 2 **Entrada:** qual informação ele precisa receber?
- 3 **Regra:** qual comparação será feita? O que acontece quando é verdadeira ou falsa?
- 4 **Saída:** o que deverá ser mostrado?
- 5 **Exemplos:** escolha dois valores e registre a recomendação esperada.
- 6 **Limites conhecidos:** o que deveria acontecer com zero, um número negativo ou o texto “meia hora”?

Você não precisa resolver os últimos casos. Reconhecer que o comportamento ainda não foi definido já faz parte da programação.

Uma descrição possível seria:

O Planejador ajuda uma pessoa que possui algum tempo para estudar ou construir, mas não decidiu o tamanho da próxima ação. Ele recebe a quantidade de minutos. Se o valor for menor que 20, recomenda uma alteração pequena; caso contrário, recomenda construir e testar uma melhoria. Ainda não definimos entradas negativas ou textos.

Sua resposta pode usar outras palavras. Você também pode discordar do limite. Essa discordância não é erro de programação; é uma decisão sobre o comportamento desejado.

O programa aceitará zero? Um número negativo? O texto “meia hora”? Não precisamos resolver esses casos agora. Registrá-los separa uma decisão adiada de uma decisão esquecida.

Se usar IA, faça sua descrição primeiro. Depois, peça que ela aponte até três situações ainda não definidas sem escrever código. A ferramenta pode encontrar um caso que você não considerou, mas ele não vira requisito automaticamente. Compare a sugestão com a descrição original: se “recomendar uma próxima ação” virou “criar um sistema completo de produtividade”, a resposta preservou pouco da intenção.

Se a IA disser que “até 20” e “menos de 20” são equivalentes, confronte a afirmação com o caso exato de 20. Verificar também é transformar linguagem vaga em exemplo observável.

Autonomia não significa memorizar tudo

Desenvolvedores consultam documentação, exemplos, anotações e código existente. A pergunta útil não é se você lembra toda a sintaxe, mas:

Consigo reconstruir esta solução entendendo as decisões, mesmo que precise consultar como representá-las?

Existe diferença entre pesquisar como escrever uma condição cujo comportamento você já definiu e pedir que outra pessoa — ou uma IA — decida toda a condição por você.

Autonomia não elimina ferramentas. Ela muda sua relação com elas.

Depois da atividade, uma IA pode revisar sua descrição e apontar ambiguidades. Ela não deve decidir automaticamente qual problema você pretende resolver, qual regra considera correta e o que entra nesta versão.

Prever comportamento é diferente de reconhecer código

Quando a resposta já está na tela, é fácil sentir que ela era óbvia. Por isso, volte à regra e produza exemplos que ainda não executou.

Se a entrada for 18, qual mensagem deveria aparecer? E com 20? Se decidirmos que “sessão curta” significa até 20 minutos, qual comportamento muda?

Essas perguntas não testam memória. Verificam se você consegue partir de uma definição e deduzir consequências.

Também ajudam a distinguir dois problemas:

- **não sei representar a regra em JavaScript:** a decisão está clara, mas falta consultar a sintaxe;
- **não sei qual regra quero:** o problema está na definição do comportamento, não na linguagem.

Pedir código para resolver a segunda situação costuma esconder a indecisão em vez de resolvê-la.

O que você leva deste capítulo

Descreva o Planejador para alguém que não viu o código: o problema, a entrada, a comparação, os dois comportamentos atuais e um caso ainda não definido. Depois, explique o que $25 < 20$ pergunta e como uma expressão válida poderia representar a regra errada.

Consulte o capítulo se necessário. O importante é reconstruir a relação entre intenção, comportamento e código, não repetir os termos de memória.

No primeiro capítulo, você executou e alterou uma pequena regra. Neste, separou a regra do código e definiu o comportamento antes da implementação.

Essa separação será nossa referência quando o programa crescer. Sempre que os símbolos esconderem a intenção, voltaremos à pergunta: qual entrada chega, qual decisão deveria acontecer e qual resultado será considerado correto?

O próximo passo é identificar quais fundamentos ajudam você a controlar essa decisão — e quais assuntos podem esperar.

O que merece sua atenção agora

Você já executou uma regra e separou o código do comportamento que ele representa. Agora aparece outra dúvida:

O que preciso aprender para continuar?

Uma busca pode devolver lógica, estruturas de dados, orientação a objetos, Git, banco de dados, APIs, redes, testes, arquitetura, cloud, Docker, segurança e inteligência artificial — tudo com aparência de urgência.

Um roadmap pode mostrar um território. Ele não decide qual deve ser seu próximo passo.

Para continuar construindo o Planejador, precisamos de quatro elementos que já aparecem no problema:

- valores;
- variáveis;
- condições;
- entrada e saída.

Eles não estão aqui apenas porque costumam aparecer em cursos introdutórios. Estão aqui porque sustentam o próximo comportamento do projeto.

Prioridade não é importância absoluta

Um assunto merece prioridade agora quando ajuda a compreender o próximo comportamento, é dependência para o passo seguinte ou pode ser praticado com feedback rápido. Esses critérios não medem a importância absoluta do tema; separam necessidade atual de ansiedade produzida por listas enormes.

Valores, variáveis e condições atendem a esses critérios. Docker pode ser importante em outros projetos, mas não explica por que `10 < 20` produz `true`. Um framework pode acelerar uma interface, mas acrescentaria convenções antes de controlarmos a regra principal.

A ordem de estudo deve nascer do problema atual, não da quantidade de assuntos colocados em um diagrama.

O feijão com arroz

Aprender o “feijão com arroz” não significa memorizar uma linguagem inteira antes de construir. Significa reconhecer os elementos básicos usados no código e compreender o papel de cada um.

Eu já avancei para uma ferramenta antes de entender bem a camada que ela facilitava. Durante um período, usava jQuery para manipular páginas, resolver o que precisava e criar algumas gambiarras. Achava mais fácil e rápido do que fazer as mesmas coisas diretamente com JavaScript.

O problema aparecia quando eu precisava sair daquele caminho conhecido. Se me pedissem a mesma tarefa apenas com JavaScript, eu tremia na base. Mais tarde, ao trabalhar com React, precisei voltar e estudar JavaScript porque as lacunas ficaram evidentes.

A conclusão não é que usar bibliotecas seja errado. jQuery me permitiu construir e resolver problemas reais. A questão é perceber quando uma ferramenta acelera algo que você compreende e quando esconde uma parte que ainda precisa aprender.

A IA amplia essa diferença. Por isso, não vamos evitar ferramentas; vamos reduzir temporariamente o número de camadas entre você e o comportamento do programa.

Valores: as informações do programa

Um valor é uma informação com a qual o programa trabalha.

Valor	Tipo reconhecido agora	Papel no Planejador
10 e 20	Número	Representar minutos disponíveis
"Faça uma alteração pequena."	Texto	Representar uma recomendação
true e false	Booleano	Representar o resultado de uma condição

Você não precisa estudar todos os tipos de valor existentes em JavaScript. Precisa reconhecer os que participam do comportamento atual.

Variáveis: nomes para valores importantes

No console, digite:

JAVASCRIPT

```
let minutos = 10
```

Essa instrução cria uma variável chamada `minutos` e associa a ela o valor 10.

- `let` informa que estamos criando uma variável cujo valor poderá mudar;
- `minutos` é o nome escolhido;
- `=` atribui o valor que aparece à direita;
- `10` é o valor inicial.

O console pode mostrar `undefined`. Nesta atividade, isso não significa falha: a instrução criou a variável, mas não produziu uma saída para exibição. Antes de corrigir uma palavra inesperada, observe se a execução realmente foi interrompida.

Digite `minutos` e o console mostrará 10.

Compare:

JAVASCRIPT

```
10 < 20  
minutos < 20
```

As expressões podem produzir o mesmo resultado, mas a segunda revela o significado do primeiro valor. `minutos` é mais informativo do que nomes como `x` ou `valor1`.

Para alterar a variável, digite:

JAVASCRIPT

```
minutos = 35
```

Não usamos `let` novamente porque a variável já existe. Estamos apenas mudando seu valor.

Se o navegador informar que `minutos` já foi declarado, atualize a página em branco ou altere o valor sem repetir `let`. Você tentou criar novamente um nome que já existia naquela sessão.

Atribuição não é comparação

Na matemática, `=` costuma expressar igualdade. Aqui:

JAVASCRIPT

```
minutos = 35
```

significa que a variável `minutos` passa a receber o valor 35.

Já a expressão:

JAVASCRIPT

```
minutos < 20
```

pergunta se o valor associado a `minutos` é menor que 20.

Neste momento, você precisa reconhecer apenas:

- `=` atribui um valor;

- `<` compara se um valor é menor que outro.

Outros operadores aparecerão quando um comportamento realmente precisar deles.

Se esquecer um símbolo, consulte documentação ou uma explicação. O critério não é lembrar cada operador sem apoio. É saber qual relação precisa representar e verificar se o símbolo consultado produz essa relação nos casos relevantes.

Condições orientam decisões

Uma condição é uma expressão avaliada como verdadeira ou falsa.

Com `minutos` valendo 35, `minutos < 20` produz `false`. Depois de executar `minutos = 10`, a mesma condição produz `true`.

A regra permaneceu igual; a entrada mudou.

A mesma regra pode produzir resultados diferentes quando recebe valores diferentes.

O booleano não é uma opinião do computador sobre a qualidade da sessão. Ele responde apenas à pergunta exata que escrevemos. Se a pergunta é `minutos < 20`, o resultado informa se o valor atual satisfaz essa relação.

“Tenho pouco tempo?” pode depender de contexto. `minutos < 20` utiliza um limite explícito. Programar exige transformar parte da ambiguidade do problema em decisões executáveis.

Mais tarde, a condição escolherá entre textos. Por enquanto, separar a comparação da mensagem evita aprender duas estruturas novas ao mesmo tempo. O nome `minutos` também ajuda: ele não altera a comparação, mas revela a intenção que um nome como `x` esconderia.

Entrada, regra e saída

Agora podemos relacionar os fundamentos ao programa:

- **entrada:** o valor atribuído a `minutos`;
- **regra:** `minutos < 20`;

- **saída atual:** true ou false;
- **saída futura:** uma das mensagens do Planejador.

Ainda não existe formulário ou botão. Essas interfaces acrescentariam conceitos que a decisão principal não exige.

ATIVIDADE

controle a regra

Atualize a página em branco para iniciar uma sessão limpa. Digite:

JAVASCRIPT

```
let minutos = 12
```

Antes de executar `minutos < 20`, registre sua previsão. Depois execute e observe.

Repita o processo alterando apenas o valor de minutos:

JAVASCRIPT

```
minutos = 20
minutos < 20

minutos = 21
minutos < 20
```

Registre:

minutos	Previsão	Resultado observado	Explicação
12			
20			
21			

O caso 20 está exatamente na fronteira. Não preencha apenas com true e false: explique por que ele produz esse resultado.

Altere um valor por vez, registre a previsão, execute e explique qualquer divergência antes da próxima mudança. Assim você preserva a informação sobre o que produziu o novo resultado.

Depois escolha outro valor, faça a previsão, execute e verifique. Essa pequena escolha própria mostra se você está usando a regra ou apenas repetindo a tabela.

Decompor é reduzir o problema

“Planejar uma sessão” poderia envolver energia, matéria, prazo, dificuldade e histórico. Nosso programa considera poucos dados para que possamos observar uma decisão de cada vez.

Podemos decompor o comportamento em:

- 1 receber os minutos;
- 2 compará-los com um limite;
- 3 escolher uma recomendação;
- 4 exibir o resultado.

Até agora, trabalhamos com as duas primeiras partes. Decompor não significa criar tarefas artificiais; significa encontrar uma parte pequena que produza feedback real.

Uma ideia ampla pode ser dividida de várias maneiras. Aqui, isolar a comparação permite compreender a decisão antes de adicionar mensagens e a estrutura que escolherá entre elas. Cada parte nova precisa justificar o que acrescenta ao comportamento.

Funções e repetição: reconheça antes de aprofundar

Durante a atividade, você repetiu a condição para valores diferentes. Uma **função** permite dar nome a um comportamento e executá-lo com outras entradas. Ela será útil quando o Planejador precisar escolher uma recomendação várias vezes.

Um **laço de repetição** também permite repetir instruções, mas ainda não resolve um problema necessário aqui. Temos poucos casos e podemos executá-los diretamente.

Mais adiante, a função permitirá usar a mesma regra com combinações diferentes. Quando ela aparecer, você já terá sentido a repetição que ela resolve. Um laço continuaria acrescentando sintaxe sem melhorar o pequeno conjunto de testes manuais.

Essa é a diferença entre reconhecer que algo existe e precisar estudá-lo agora.

Um único mapa de prioridades

Categoria	Neste ciclo
APRENDA AGORA	Valores usados no projeto, variáveis, atribuição, condições, entrada, regra e saída
PRATIQUE AGORA	Prever, alterar valores, explicar condições e testar abaixo, acima e no limite
ENTENDA QUE EXISTE	Funções, laços, estruturas para reunir dados, APIs, bancos, Git e publicação
ESTUDE DEPOIS	Frameworks, infraestrutura, arquiteturas complexas e sistemas com várias camadas

Se tudo entrar em **APRENDA AGORA**, a classificação não resolveu o problema. Para a próxima sessão, escolha no máximo três conhecimentos e uma prática.

O mapa não precisa ficar bonito nem completo. Pegue os assuntos que disputam sua atenção e pergunte se ajudam a compreender o código atual, fazer a próxima alteração ou resolver um problema que já apareceu. Escolha no máximo três conhecimentos e uma prática para a próxima sessão. O restante pode esperar sem ser considerado inútil.

Se usar IA, peça explicação de um símbolo ou mensagem específica depois de fazer sua previsão. Não entregue a ela a escolha da regra nem sua explicação final. Verifique sempre executando os valores relevantes.

Uma pergunta adequada seria:

Explique `let minutos = 12` para uma pessoa que nunca programou. Não continue o projeto. Depois, faça uma pergunta para verificar se eu entendi a diferença entre o nome e o valor.

Se a ferramenta produzir uma solução maior, ignore o que não resolve a dúvida atual.

Antes de seguir, confirme que consegue identificar os valores do `Planejador`, explicar `minutos`, distinguir atribuição de comparação e prever a condição abaixo, acima e no limite. Também explique por que uma função será útil no próximo avanço e um laço ainda não.

Se algo ainda parece mágico, transforme-o em uma pergunta específica. “Não entendi variáveis” é amplo. “Não entendi por que escrevo `let` apenas quando crio o nome” já indica o que observar e pesquisar.

O objetivo não é eliminar todas as dúvidas. É não carregar uma dúvida indispensável de forma invisível. Uma dúvida sobre a regra central merece investigação agora; uma dúvida sobre publicação pode esperar. **ENTENDA QUE EXISTE** preserva o contexto sem interromper a construção.

Você não precisa dominar JavaScript. Precisa controlar os poucos elementos que já participam do problema.

Esse controle ainda é limitado e localizado. Saber explicar `minutos < 20` não significa dominar condições em qualquer sistema. Significa que existe uma base concreta para acrescentar a próxima estrutura.

No próximo capítulo, veremos por que fechar o ciclo entre previsão, execução, explicação, alteração e teste produz mais informação do que apenas acompanhar uma solução pronta.

Aprender é fechar ciclos

No capítulo anterior, você usou valores diferentes para observar a mesma condição. Não apenas leu sobre variáveis: fez previsões, executou expressões e comparou respostas.

Essa diferença parece pequena, mas muda a qualidade do estudo.

É possível assistir a uma explicação inteira e sentir que tudo fez sentido. Também é possível acompanhar um tutorial, copiar o código corretamente e chegar ao mesmo resultado apresentado pelo autor.

O problema aparece quando você se afasta do exemplo. Consegue repetir a ideia sem seguir os mesmos passos? Explicar por que funcionou? Alterar uma regra e antecipar o efeito? Investigar uma divergência?

A sensação de familiaridade não responde a essas perguntas. Para descobrir o que está sob seu controle, precisamos fechar o ciclo entre o que esperamos, o que o programa fez e o que conseguimos fazer depois.

Consumir não produz feedback suficiente

Ao observar alguém programando, você pode reconhecer variáveis, condições e saídas. Esse reconhecimento ajuda a construir vocabulário e conhecer soluções possíveis.

Mas o problema já foi definido, a ordem já foi organizada e os erros mais demorados podem ter sido removidos. A atividade muda quando você precisa tomar uma decisão antes de ver a resposta.

Se sua previsão estiver errada, surge uma diferença concreta para investigar:

Eu esperava uma coisa, mas o programa fez outra. Onde minha interpretação se afastou do comportamento real?

O erro deixa de significar apenas “não consegui” e passa a fornecer informação.

Um exemplo pode parecer familiar porque você acabou de vê-lo. Leitura, aulas e tutoriais fornecem vocabulário e modelos; o feedback sobre sua interpretação aparece quando você alterna consumo com ação.

Executar não encerra a atividade

Considere `minutos < 20`. Você executa e recebe `true`. É tentador pensar: “funcionou, próximo assunto”.

Mas ainda podemos perguntar:

- qual era o valor de `minutos`?
- por que a comparação produziu `true`?
- o que aconteceria exatamente com 20?
- qual parte mudaria para usar outro limite?
- os casos anteriores continuariam coerentes?

Executar fornece uma resposta do computador. Aprender exige fazer alguma coisa com ela.

Um programa pode funcionar quando você ainda não compreende partes relevantes. Também pode falhar enquanto você desenvolve uma interpretação melhor. O que importa aqui é o que você consegue observar e fazer a partir da execução.

O que cada parte do ciclo revela

Usaremos a sequência:

PREVER → EXECUTAR → OBSERVAR → EXPLICAR → ALTERAR → TESTAR

Ela não deve virar uma cerimônia para cada linha. Serve para revelar lacunas que ficariam escondidas se apenas executássemos uma solução pronta.

PREVER expõe sua interpretação

Uma previsão registra o que você acredita que acontecerá antes da resposta.

Ela não precisa estar correta, mas precisa ser específica:

Com minutos igual a 20, espero false, porque 20 não é menor que 20.

“Acho que funciona” não informa qual comportamento você esperava nem por quê.

EXECUTAR confronta a previsão com o programa

O computador aplica as instruções recebidas, não aquelas que imaginamos ter escrito. Executar tira a ideia do campo das suposições, mas ainda não determina se a solução está correta. Para avaliar o resultado, precisamos de uma expectativa.

OBSERVAR separa fato de interpretação

“Deu errado” oferece pouca informação.

Com minutos igual a 20, a condição retornou false, mas eu esperava true.

Essa descrição contém valor, instrução, resultado e divergência. Agora existe um problema investigável.

EXPLICAR conecta causa e efeito

Compare:

Deu falso porque o código deu falso.

com:

A comparação verifica se 20 é menor que 20. Como os valores são iguais, `minutos < 20` produz false.

A segunda explicação relaciona a regra ao resultado. Também revela onde a compreensão termina: talvez você entenda a comparação, mas ainda precise pesquisar o símbolo usado.

ALTERAR verifica se a compreensão é utilizável

Uma alteração útil é pequena e intencional. Antes de mudar um valor, limite ou mensagem, descreva o efeito esperado.

Vou mudar o limite de 20 para 25. O valor 24 deverá entrar no grupo das sessões curtas; 25 continuará fora.

Antecipar o efeito é diferente de mexer até aparecer uma saída aceitável.

TESTAR procura consequências

Se você alterou uma regra para corrigir o caso 20 e executou apenas esse caso, ainda não sabe o que aconteceu com os outros.

Para uma condição com limite numérico, três casos costumam revelar bastante: abaixo, exatamente no limite e acima.

Isso não prova que todo comportamento possível está correto. A formulação responsável é:

Testei estes casos, e eles corresponderam ao que defini.

Quando uma alteração afeta uma região específica, escolha casos capazes de atravessá-la. Mudar o limite exige testar abaixo, no limite e acima; alterar uma mensagem exige confirmar o caminho que a devolve.

Fechar o ciclo não significa acertar

Na primeira situação, uma pessoa copia uma expressão, executa, recebe o resultado esperado e segue adiante.

Na segunda, faz uma previsão, erra, observa a diferença, revê a condição, explica o motivo e testa novamente.

A primeira terminou com acerto funcional. A segunda produziu mais evidências sobre o que a pessoa compreende.

Erros investigados podem oferecer feedback que acertos não examinados escondem. Também é possível executar muitos casos e apenas seguir uma tabela. Em algum momento, você precisa produzir uma previsão, escolher uma alteração ou explicar uma divergência com suas palavras.

O ciclo se fecha quando você relaciona o que esperava, o que aconteceu, por que aconteceu, o que mudou e como verificou a mudança.

O ciclo pode terminar com uma dúvida, desde que ela esteja mais específica. Fechar o ciclo não é declarar o assunto encerrado; é obter informação suficiente para decidir o próximo passo.

Uma nova decisão para o Planejador

Até agora, o Planejador considerava apenas o tempo. Vamos acrescentar uma segunda informação: se você está travado.

Neste projeto, estar travado significa não saber qual ação executar em seguida ou enfrentar um problema que impede continuar.

Usaremos um booleano:

JAVASCRIPT

```
let travado = false
```

A regra provisória será:

Se estou travado ou tenho menos de 20 minutos, devo reduzir o próximo passo. Caso contrário, posso construir e testar uma melhoria.

Em JavaScript, `||` representa “ou”:

JAVASCRIPT

```
travado || (minutos < 20)
```

A expressão será `true` quando pelo menos uma destas condições for verdadeira:

- `travado` é `true`;
- `minutos` é menor que 20.

Por enquanto, ainda observaremos um booleano. No Capítulo 6, refinaremos esse resultado em três mensagens, dando prioridade específica ao bloqueio.

ATIVIDADE

feche um ciclo completo

Comece com uma sessão limpa no console:

JAVASCRIPT

```
let minutos = 15
let travado = false
```

Antes de executar, preveja e explique em uma frase:

JAVASCRIPT

```
travado || (minutos < 20)
```

Depois execute e compare. Repita com estas duas combinações, alterando apenas os valores:

JAVASCRIPT

```
minutos = 40
travado = true
```

JAVASCRIPT

```
minutos = 40
travado = false
```

Registre:

Minutos	travado	Previsão	Resultado observado	Explicação
15	false			
40	true			
40	false			

Se alguma previsão divergir, observe as partes separadamente:

JAVASCRIPT

```
travado
minutos < 20
travado || (minutos < 20)
```


Você terá três observações: o valor de travado, o resultado da comparação e o resultado combinado. Separar um problema em partes observáveis é uma forma inicial de depuração.

Formule uma hipótese antes de corrigir:

Acho que a expressão produziu `true` porque travado ainda estava verdadeiro.

Executar apenas travado confirma ou contraria a hipótese. Se ele for `false`, observe `minutos < 20`. Não altere variáveis e limite ao mesmo tempo: a nova saída deixaria de indicar qual mudança resolveu a divergência.

Agora altere o requisito:

A sessão deve ser reduzida quando estou travado ou tenho menos de 25 minutos.

Antes de editar, defina os resultados esperados:

Minutos	travado	Resultado esperado
24	<code>false</code>	<code>true</code>
25	<code>false</code>	<code>false</code>
40	<code>true</code>	<code>true</code>
40	<code>false</code>	<code>false</code>

Identifique a parte da expressão que representa o limite atual e altere somente ela. Depois execute os quatro casos.

Se um resultado divergir, observe cada condição e formule uma hipótese. Ao concluir, explique por que o novo requisito exigiu trocar 20 por 25 e registre uma coisa que já consegue alterar e outra que ainda precisa consultar.

O ciclo é uma síntese, não uma invenção

A sequência deste livro não é apresentada como método científico original do MarllonCode. Ela organiza práticas anteriores de ensino e aprendizagem.

Uma referência próxima é o PRIMM: **Predict, Run, Investigate, Modify e Make**. A abordagem enfatiza leitura e compreensão antes de exigir que o estudante construa tudo sozinho. Um estudo clássico sobre autoexplicação, realizado com estudantes que analisavam exemplos resolvidos de problemas de mecânica, relacionou explicações geradas pelos próprios estudantes a diferenças na compreensão e no uso posterior dos exemplos.^{1 2}

Essas pesquisas ocorreram em contextos específicos e não demonstram que repetir seis palavras garanta aprendizagem, nem que esta sequência seja superior a toda alternativa. O valor editorial está em organizar práticas reconhecíveis de forma simples para orientar o projeto.

A recomendação do livro é mais limitada:

Alternar previsão, execução, investigação, explicação, modificação e teste cria oportunidades para revelar diferenças entre o que você acredita que o programa faz e o que ele realmente faz.

O ciclo é uma estrutura de prática, não uma certificação de aprendizagem.

Seu valor aparece quando muda sua ação diante do código: em vez de apenas receber a saída, você produz uma expectativa, investiga a diferença e escolhe uma verificação relacionada.

Quando a IA interrompe ou fortalece o ciclo

Imagine que você não compreendeu travado || (minutos < 20).

Você pode pedir “corrija e entregue o código completo”. Talvez receba uma versão funcional, mas sua previsão e sua hipótese continuam ausentes.

Ou pode informar:

Eu esperava `false`, mas recebi `true`. `travado` vale `true` e `minutos` vale 40. Explique qual parte da minha interpretação está errada sem reescrever o programa.

A segunda pergunta preserva o problema que você precisa resolver. A explicação ainda deve ser comparada com a execução.

Usar IA com frequência não é automaticamente dependência. O sinal de perda de controle aparece quando você deixa de conseguir dizer o que esperava, o que mudou ou qual teste faria em seguida.

O ciclo não precisa virar formulário

Com o tempo, algumas etapas acontecem mentalmente em poucos segundos:

Espero `false`. Recebi `true`. `travado` ainda está verdadeiro. Vou alterar e testar de novo.

Vale tornar o ciclo explícito quando o conceito é novo, a previsão falha, uma mudança afeta vários casos, o programa funciona sem você saber por quê ou uma solução gerada por IA ainda parece uma caixa-preta.

Quando o comportamento é conhecido e de baixo risco, o ciclo pode acontecer rapidamente. Você prevê mentalmente, executa, percebe uma divergência, identifica o valor incorreto e testa de novo. A estrutura deve aumentar clareza, não criar trabalho vazio.

Antes de seguir, confirme que consegue separar esperado e observado, investigar as duas partes de `||` e alterar o limite sem mexer no restante. Você não precisa acertar todas as previsões; precisa usar a divergência para escolher o próximo passo.

Quando uma divergência relevante aparecer, guarde uma nota curta:

Etapa	Registro possível
Previsão	“Espero <code>true</code> porque 15 é menor que 20.”
Observação	“Recebi <code>false</code> .”
Hipótese	“Talvez <code>travado</code> ou <code>minutos</code> não tenha o valor que imaginei.”
Verificação	“Observei as partes separadamente.”

Não use a tabela para todas as linhas. Use-a quando perceber que está prestes a modificar o código sem saber por quê. Se a previsão estiver correta, explique qual parte da regra sustenta o acerto.

No próximo capítulo, aplicaremos esse critério diretamente à IA: quando ela amplia a investigação e quando começa a substituir o controle que você ainda precisa desenvolver.

IA como apoio, não como substituição

No capítulo anterior, fechamos o ciclo entre o que esperamos, o que o programa faz e o que conseguimos explicar, alterar e testar.

Agora precisamos decidir onde a IA entra nesse ciclo.

No Capítulo 3, contei que usei jQuery antes de compreender bem JavaScript. A ferramenta acelerava tarefas, mas também escondia uma lacuna que reapareceu quando cheguei ao React.

O problema não era a ferramenta. Era permanecer confortável na abstração sem recuperar o que ela escondia.

A IA reduz ainda mais a distância entre intenção e resultado. Você descreve uma função e recebe código; algumas ferramentas também investigam arquivos, propõem mudanças e executam comandos.

A questão não é se você deveria usar essa capacidade. É como utilizá-la sem remover continuamente as ações que precisa praticar para aprender.

Resultado, aprendizagem e controle

Imagine que uma IA escreveu uma função, você a executou e a saída parece correta.

Isso demonstra um resultado. Ainda não sabemos quanto você aprendeu nem quanto controle possui.

Dimensão	Pergunta
Resultado	O programa executou e produziu a saída esperada?
Aprendizagem	Você consegue explicar por que o resultado aconteceu?
Controle	Você consegue alterar, testar e corrigir o comportamento?

DIAGRAMA D3

01	02	03
RESULTADO	APRENDIZAGEM	CONTROLE
O programa produziu a saída esperada?	Você consegue explicar por que aconteceu?	Você consegue alterar, testar e corrigir?

As dimensões podem aparecer juntas, mas uma não garante a outra. Você pode aprender investigando algo que ainda não concluiu e pode concluir uma tarefa sem conseguir continuar sozinho.

As pesquisas disponíveis pedem cautela, não um veredito único. Um experimento de 2023 encontrou ganhos imediatos em tarefas de autoria de código entre jovens iniciantes que usaram geração de código; a diferença posterior no pós-teste não foi estatisticamente significativa. Um preprint de 2026 encontrou menor compreensão posterior entre participantes experientes em Python que usaram IA enquanto aprendiam uma biblioteca desconhecida, sem ganho médio significativo de eficiência. Públicos, ferramentas e tarefas eram diferentes.^{1 2}

Não podemos concluir que IA sempre ajuda ou sempre prejudica. A formulação útil para este livro é mais limitada:

Terminar a tarefa não será nossa única medida de aprendizagem.

Antes, durante e depois da resposta

Você pode usar IA desde o começo. O tipo de ajuda deve acompanhar o que pretende praticar.

Antes

Defina a entrada, o comportamento esperado e sua primeira hipótese. Se a IA decidir objetivo, regra e implementação, restarão poucas referências próprias para avaliar a resposta.

Durante

Peça ajuda que torne o raciocínio mais visível:

Não altere este código. Explique a ordem das condições e depois faça uma pergunta para verificar se entendi.

Você não precisa evitar toda geração de código. Precisa perceber quando ela elimina a decisão que pretendia praticar.

Não envie credenciais, chaves, dados pessoais ou informações sensíveis.

Depois

A resposta precisa voltar ao ambiente de execução. Execute, compare com sua previsão, teste valores diferentes e altere uma coisa por vez. Quando houver documentação oficial relevante, consulte-a.

Uma resposta bem escrita continua sendo uma hipótese até ser verificada. O próprio GitHub alerta que explicações, correções e testes gerados pelo Copilot podem estar incompletos ou incorretos.³

O que manter com você

Neste livro, três ações não serão entregues imediatamente à IA:

- 1 **A primeira previsão:** tente responder antes de perguntar qual será a saída.
- 2 **O comportamento esperado:** defina o que considera correto antes de pedir implementação.
- 3 **A verificação final:** a IA pode sugerir código e testes; a decisão de considerar o comportamento verificado continua sendo sua.

O nível de controle também depende do contexto:

Contexto	Controle necessário
Exploração descartável	Reconhecer que o resultado é provisório e pode estar errado
Projeto de aprendizagem	Explicar, alterar e testar o conceito estudado
Projeto mantido	Inspecionar mudanças, testar comportamentos e avaliar impacto
Sistema com usuários ou dados importantes	Revisão proporcional ao risco e responsabilidade humana definida

Quanto maior a consequência, maior deve ser a revisão antes de aceitar uma alteração.

Vibe coding depende do que está em jogo

O termo *vibe coding* é usado com sentidos diferentes. Pode descrever a construção por conversa com uma IA ou, em sentido mais extremo, a aceitação contínua de sugestões sem acompanhar o código.

Em vez de decidir se o termo inteiro é bom ou ruim, pergunte:

- isto é um experimento ou algo que precisará ser mantido?
- existem usuários, dinheiro, credenciais ou dados envolvidos?
- quem conseguirá investigar um erro?
- consigo alterar uma regra sem pedir que a IA reconstrua tudo?

Para explorar uma ideia, delegar bastante pode ser razoável. O risco aparece quando um protótipo sem controle é tratado como produto pronto ou quando velocidade é confundida com domínio.

O mesmo código pode mudar de exigência ao mudar de contexto. Uma demonstração local pode tolerar limitações registradas; uma ferramenta usada por clientes precisa de revisão, segurança e manutenção proporcionais às consequências.

Um diagnóstico feito com IA

No desenvolvimento da *Conjuga*, um formulário começou a retornar erro 500 em produção.

O erro aparecia na interface, mas a causa poderia estar em outra camada. Usamos o Codex para restringir a investigação aos arquivos relevantes e mapear o caminho entre formulário, rota da API, serviço, repositório, banco e variáveis de ambiente.

A investigação revelou problemas diferentes ao longo do caminho: configuração apontando para um projeto antigo, tabela ausente e permissões insuficientes.

O Codex ajudou a interpretar erros e organizar hipóteses. Eu criei o novo projeto no Supabase, atualizei as variáveis na Vercel, executei a migração, corriji as permissões e testei novamente a tabela, a inserção e o formulário.

A IA acelerou a investigação. A compreensão veio da sequência: formular uma hipótese, executar uma verificação, observar e decidir o próximo passo.

É uma experiência concreta de IA ampliando uma investigação sem eliminar a verificação, não uma regra universal.

ATIVIDADE

avale uma resposta plausível

Considere:

JAVASCRIPT

```
let minutos = 20
let travado = false

travado || (minutos < 20)
```

Sem executar e sem consultar IA, responda:

- `minutos < 20` será verdadeiro ou falso?
- qual será o resultado da expressão completa?
- o que mudaria se `travado` fosse `true`?

Agora avalie esta explicação, como se tivesse sido recebida de uma IA:

“A expressão retorna true quando a pessoa está travada ou possuí até 20 minutos.”

Ela parece plausível, mas está incorreta. < representa “menor que”, não “até”. Com exatamente 20 minutos e travado igual a false, as duas partes são falsas.

Não aceite essa conclusão apenas porque o livro afirmou. Verifique no console:

- 1 execute a expressão com 20 minutos;
- 2 teste 19 e 21;
- 3 altere travado para true;
- 4 explique qual parte determinou cada resultado.

Se tiver IA disponível, peça que ela critique a explicação sem mudar o código. Depois compare a nova resposta com as execuções.

O objetivo é praticar uma habilidade específica: reconhecer uma resposta convincente, localizar a afirmação verificável e voltar ao comportamento real.

Um protocolo para usar quando fizer diferença

USE IA PARA

- receber outra explicação depois de formular sua hipótese;
- interpretar uma mensagem de erro específica;
- sugerir casos adicionais;
- comparar alternativas sem decidir automaticamente por você.

NÃO TERCEIRIZE

- sua primeira previsão;
- o comportamento esperado;
- a explicação final com suas palavras;
- a escolha do que pertence ao escopo;

- a verificação do resultado.

VERIFIQUE

- executando o código;
- comparando esperado e observado;
- consultando documentação quando necessário;
- confirmando que consegue explicar e alterar a parte estudada.

Se você não consegue dizer o que o código deveria produzir, alterar um valor simples ou escolher um teste, reduza temporariamente o que delega. Recupere uma parte pequena e só então decida como continuar.

Quando recuperar o controle

Sinais de dependência incluem:

- você não sabe quais arquivos ou linhas foram modificados;
- qualquer mudança simples exige um novo pedido completo;
- a resposta é aceita antes de ser executada;
- um erro faz você abandonar sua hipótese e pedir outro código;
- você não consegue distinguir o requisito original das sugestões acrescentadas pela ferramenta.

Recuperar controle não significa recomeçar. Escolha uma parte observável, defina o esperado, execute, explique e faça uma alteração pequena. O nível de revisão deve acompanhar as consequências: um experimento descartável tolera incertezas que seriam inadequadas em um sistema com dados de clientes.

A pergunta que permanece

A IA pode acelerar geração, pesquisa, explicações e investigação. Nenhuma dessas capacidades elimina a necessidade de decidir o que você está tentando aprender.

Se o objetivo da sessão é praticar condições, pedir imediatamente a condição pronta remove a ação central. Se o objetivo é investigar uma integração complexa que você já compreende em partes, delegar busca e

comparação de arquivos pode liberar atenção para decisões mais importantes.

Antes de consultar a ferramenta, pergunte:

Qual ação cognitiva preciso praticar nesta tarefa — e qual parte posso acelerar sem apagá-la?

Ela evita duas regras simplistas: “nunca gere código” e “deixe a IA fazer tudo”. Conforme avança, você poderá delegar tarefas diferentes. O critério é não terceirizar continuamente o que ainda precisa praticar para avaliar o próprio trabalho.

No próximo capítulo, trataremos de outra forma comum de perder o controle: tentar construir coisas demais ao mesmo tempo.

Reduza até caber

No capítulo anterior, vimos como usar IA sem entregar continuamente previsão, comportamento esperado e verificação.

Existe outra forma comum de perder o controle: tentar construir coisas demais ao mesmo tempo.

Uma ideia como “criar um programa para organizar minha próxima sessão” pode crescer até incluir assunto estudado, dificuldade, histórico, calendário, notificações, interface e recomendações com IA.

As possibilidades não são absurdas. O problema é transformá-las em pré-requisitos antes de concluir a primeira versão.

Neste capítulo, não escreveremos o programa completo. Vamos decidir exatamente o que precisa ser construído — e o que ficará de fora.

O que significa caber

Reduzir não é apenas diminuir linhas. Uma versão precisa caber:

- **no tempo:** deve avançar em sessões reais;
- **nos conhecimentos disponíveis:** pode introduzir algo novo, mas não várias camadas antes do primeiro resultado;
- **no ambiente:** cada ferramenta, instalação e serviço acrescenta fricção;
- **na capacidade de verificar:** você precisa conseguir prever, explicar, alterar e testar os comportamentos assumidos.

Para um projeto de aprendizagem, a capacidade de verificar é decisiva. Uma IA pode gerar um sistema grande em segundos; isso não reduz automaticamente o custo de compreendê-lo. Pergunte se o projeto cabe nas sessões que você realmente possui, quantos conceitos e ferramentas

aparecem antes do primeiro resultado e se consegue definir quando a versão está correta para o escopo escolhido.

O objetivo não é construir apenas coisas fáceis. É aumentar a dificuldade de maneira que cada mudança ainda produza feedback compreensível.

Uma versão pequena precisa produzir algo

Criar a pasta, escolher a linguagem e iniciar o projeto são preparações. Uma vitória funcional exige um comportamento concluído.

No nosso caso:

A pessoa informa quanto tempo possui e se está travada. O programa escolhe e exibe uma recomendação para a próxima sessão.

Não haverá conta, histórico, calendário ou interface. Mesmo assim, o programa receberá informações, aplicará regras e produzirá uma saída verificável.

Comece pelo problema, não pelo produto imaginado

“Quero criar um aplicativo de produtividade com IA” descreve uma categoria de solução. O problema ainda está escondido.

Para o Planejador:

Uma pessoa possui algum tempo para estudar ou construir, mas não consegue decidir o tamanho da próxima ação.

Essa formulação permite perguntar:

Sem esta funcionalidade, o programa ainda consegue reduzir o problema definido?

Se a resposta for sim, ela provavelmente pode esperar.

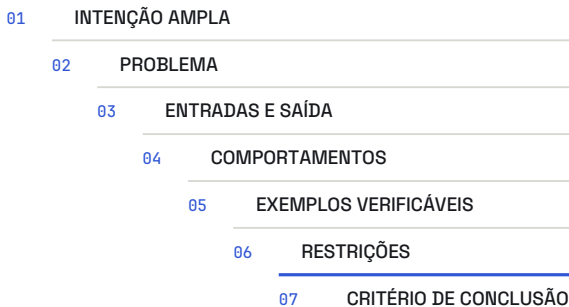
O usuário inicial será o próprio leitor, em um ambiente controlado. Isso significa que não precisamos criar contas, armazenar preferências nem distribuir o projeto pela internet.

Todo programa atende alguém, mesmo que seja a pessoa que o escreveu. Como você mesmo alterará valores antes de executar, não precisamos criar uma interface nem preparar a entrada para um público desconhecido.

Isso não descreve a versão ideal de um produto comercial. Descreve a menor situação em que o núcleo pode ser construído e verificado.

Construa a especificação em uma única progressão

DIAGRAMA D4



Começaremos pelo menor núcleo verificável e acrescentaremos apenas o necessário.

1. Entradas e saída

O programa receberá:

- minutos: tempo disponível;
- travado: existência de um bloqueio.

A saída será uma entre três mensagens. Temos, portanto:

Duas entradas, decisões ordenadas e uma saída.

2. Comportamentos obrigatórios

Escrever as mensagens antes do código impede que a implementação escolha palavras e prioridades por acidente. As frases fazem parte do comportamento observável.

Existe um bloqueio

Se travado for `true`, exibir:

Explique o que você já tentou.

O bloqueio terá prioridade sobre o tempo.

Há pouco tempo e nenhum bloqueio

Se travado for `false` e minutos for menor que 20, exibir:

Faça uma alteração pequena.

Há tempo suficiente e nenhum bloqueio

Se travado for `false` e minutos for igual ou maior que 20, exibir:

Construa e teste uma melhoria.

3. Exemplos verificáveis

Minutos	travado	Resultado esperado
10	false	Faça uma alteração pequena.
20	false	Construa e teste uma melhoria.
40	false	Construa e teste uma melhoria.
10	true	Explique o que você já tentou.
40	true	Explique o que você já tentou.

Com exatamente 20 minutos, a primeira condição de tempo é falsa.

Quando travado for `true`, a mensagem do bloqueio vence mesmo com 40 minutos disponíveis.

4. Restrições conhecidas

Nesta versão, minutos será um número igual ou maior que zero e travado será true ou false. Nós mesmos alteraremos as entradas no código.

Não trataremos número negativo, texto como "meia hora" nem ausência de valor. Existe diferença entre esquecer um caso e deixá-lo conscientemente fora do escopo.

Um programa usado por outras pessoas provavelmente precisaria validar entradas. Nesta versão, a restrição será conhecida e registrada. Se alguém perguntar se o Planejador aceita texto, a resposta honesta é “ainda não; este ciclo trabalha com valores controlados”.

Essa é uma limitação explícita da versão atual, não um caso esquecido.

5. O que fica de fora

A versão não terá interface gráfica, armazenamento, autenticação, calendário, notificações, pacotes, APIs, banco de dados, frameworks, IA integrada nem publicação na internet.

Também não precisará de arrays, objetos, laços, classes, módulos ou assincronismo.

Essa exclusão não empobrece o núcleo. Ela remove camadas que ainda não são necessárias para receber valores, tomar uma decisão e exibir uma mensagem.

Sem banco de dados, o programa não preservará sessões. Sem interface, as entradas serão alteradas no arquivo. Sem IA integrada, as recomendações virão das três regras. Cada escolha reduz dependências e deixa o núcleo visível.

6. Critério de conclusão

A versão estará funcionalmente concluída quando:

- possuir valores para minutos e travado;
- escolher e exibir uma das três mensagens;
- respeitar a prioridade do bloqueio;
- produzir os resultados definidos nos casos principais;
- permitir uma alteração intencional e nova verificação.

O controle será aprofundado no Capítulo 8. Por enquanto, precisamos chegar a uma versão que execute sem esconder sua especificação.

O critério também diz quando parar. Depois que os três comportamentos funcionarem, uma quarta recomendação inicia outra versão. Defeitos dentro do escopo ainda precisam ser corrigidos; novas funcionalidades podem esperar. O Capítulo 7 buscará a conclusão funcional, e o 8 pedirá evidências de controle.

ATIVIDADE

feche o escopo

Antes de continuar, escreva um único parágrafo com o problema, as duas entradas, a saída e os três comportamentos. Abaixo dele, registre três combinações de entrada e resultado esperado — incluindo exatamente 20 minutos e um bloqueio —, algumas ideias que ficarão de fora e a frase “considerarei esta versão concluída quando...”.

Não preencha a ficha como uma prova de terminologia. Se preferir, descreva o comportamento como uma pequena história:

Informo os minutos e se estou travado. Se existe bloqueio, o programa recomenda explicar o que já tentei. Sem bloqueio, ele escolhe entre uma alteração pequena e uma melhoria conforme o tempo.

Depois transforme a história em casos. Um caso deve verificar pouco tempo, outro o limite e outro a prioridade do bloqueio. Os exemplos são a ponte entre uma descrição geral e algo que poderá ser executado.

Depois compare sua resposta com a especificação canônica ao final do capítulo. Se adicionou várias funcionalidades, pergunte se alguma é necessária para os três comportamentos.

Se removeu tanto que não existe uma saída observável, o escopo ficou pequeno demais para produzir uma vitória. “Criar o arquivo” não resolve o problema; escolher e exibir a mensagem resolve uma parte dele.

O núcleo escondido em ideias grandes

Muitos “projetos de portfólio” são grupos de projetos. Um e-commerce completo reúne catálogo, carrinho, estoque, pagamento e pedidos; se a prática atual é totalização, o núcleo pode ser receber valores e calcular um total. Um dashboard com API, autenticação e gráficos pode começar por receber uma métrica e classificá-la. Um clone de tarefas pode começar por alterar uma tarefa entre dois estados.

Reduzir não significa construir uma versão insegura e chamá-la de pronta. Um sistema de autenticação, por exemplo, envolve responsabilidades que não desaparecem porque o formulário apareceu. Às vezes, reduzir significa escolher outro comportamento para estudar.

Essa diferença importa para projetos de portfólio. Um e-commerce “completo” mal compreendido não demonstra automaticamente mais capacidade do que uma ferramenta menor que você consegue explicar, testar e evoluir. O tamanho visual do projeto não é uma medida confiável de aprendizagem.

Em trabalho profissional, reduzir pode significar entregar primeiro o fluxo mais importante sem fingir que segurança ou manutenção deixaram de existir. O contexto muda a responsabilidade, não a necessidade de encontrar o núcleo.

Escopo não é lista de desejos

Uma lista de funcionalidades responde “o que seria interessante adicionar?”. Um escopo responde “o que precisa existir para considerar esta versão concluída?”.

Gerar novas funcionalidades com IA custa pouco. Compreender, testar e manter continua tendo custo. Por isso, velocidade de produção de código não determina o tamanho adequado do escopo.

Quando funcionalidades entram antes da primeira execução, surgem tecnologias e qualquer alteração passa a afetar partes que você não controla. Fechar o escopo impede que cada possibilidade vire obrigação.

Isso não impede revisão. Se durante a implementação descobrirmos que um comportamento obrigatório exige uma decisão ausente, a especificação pode ser corrigida. A diferença é revisar por necessidade

observada, não ampliar porque uma nova ideia parece interessante. Toda mudança precisa dizer qual problema resolve e quais testes serão afetados.

Se sua versão depende de muitas instalações, contas, tecnologias ou comportamentos que você não consegue descrever, volte à pergunta:

Dadas estas entradas, o que deve acontecer?

Mantenha as ideias removidas numa lista curta de “depois”. Ela libera sua atenção; não cria outro roadmap.

Para o Planejador, a lista poderia conter receber valores sem editar o código, validar entradas, guardar sessões anteriores e personalizar o limite. Nenhum desses itens será implementado agora.

Depois da versão funcional, cada ideia poderá ser avaliada como um novo ciclo: qual problema acrescenta, quais conceitos exige e como será verificada. “Depois” é uma área de espera, não uma promessa.

Sinais de escopo excessivo são:

- o primeiro resultado só aparece depois de várias etapas de infraestrutura;
- o critério de conclusão continua sendo “terminar o aplicativo”;
- você não sabe quais funcionalidades podem ser removidas;
- qualquer modificação exige que a IA reorganize o projeto inteiro;
- a utilidade só surgirá depois de várias versões futuras;
- você não consegue descrever os comportamentos obrigatórios sem citar tecnologias.

Uma ferramenta mais poderosa não resolve essa ausência de fronteira. Volte ao problema e formule um comportamento observável.

Se usar IA, peça que ela revise ambiguidades da especificação sem escrever código nem sugerir funcionalidades. Você continua responsável por escolher o problema, o núcleo, as exclusões e o critério de conclusão.

Uma solicitação possível é:

Revise esta especificação sem escrever código. Aponte ambiguidades que impediriam verificar os três comportamentos. Não sugira funcionalidades novas, a menos que sejam indispensáveis ao escopo existente.

Aceite uma sugestão somente se conseguir relacioná-la a um comportamento obrigatório e verificá-la com os conhecimentos disponíveis.

Especificação canônica do Planejador

Problema: ajudar o próprio leitor a escolher o tamanho da próxima ação de estudo ou construção.

Usuário: o próprio leitor, em ambiente controlado.

Entradas: minutos disponíveis e estado de bloqueio.

Saída: uma entre três mensagens.

Ordem dos comportamentos:

- 1 Se estiver travado, exibir `Explique o que você já tentou.`
- 2 Caso contrário, se possuir menos de 20 minutos, exibir `Faça uma alteração pequena.`
- 3 Nos demais casos, exibir `Construa e teste uma melhoria.`

Restrições: minutos será um número válido e não negativo; travado será `true` ou `false`; as entradas serão alteradas diretamente no código.

Exclusões principais: interface, armazenamento, autenticação, serviços externos e IA integrada.

Critério de conclusão: o programa escolhe e exibe a mensagem esperada nos três comportamentos e continua coerente depois de uma alteração intencional verificada.

Leia a especificação sem pensar em JavaScript. Você consegue prever a mensagem para qualquer combinação válida de minutos e travado? Se não, ainda existe ambiguidade. Se consegue, o próximo desafio é representar essa ordem em código sem aumentar o escopo.

Agora o escopo está fechado. No próximo capítulo, transformaremos essa especificação em um programa executável, acrescentando uma decisão de cada vez.

Se surgir uma ideia durante a implementação, registre-a e continue. A especificação é a referência que permitirá dizer se esta versão terminou.

Construa em pequenas decisões

No capítulo anterior, fechamos a especificação do Planejador da Próxima Sessão:

- duas entradas: minutos e travado;
- três mensagens exatas;
- prioridade para o bloqueio;
- casos esperados;
- critério de conclusão.

Agora vamos transformar essas decisões em um programa executável, uma por vez. Se uma etapa não funcionar, não adicionaremos a próxima.

Do console para um arquivo

O console foi suficiente para expressões curtas. Um programa com várias linhas precisa ser editado, salvo e executado novamente.

Nesta rota, usaremos:

- uma pasta para o projeto;
- um arquivo chamado `planejador.js`;
- o Visual Studio Code para editar;
- o Node.js para executar;
- o terminal integrado para enviar o comando.

Cada peça possui um papel:

Peça	Função neste percurso
Arquivo	Guardar o código
Editor	Ler e modificar o arquivo
Terminal	Receber comandos
Node.js	Executar o JavaScript fora do navegador

Escrever no editor não envia automaticamente instruções ao Node.js. Digitar um comando no terminal não modifica o arquivo. O navegador dos capítulos anteriores também não participa desta execução.

O caminho será sempre: editar o arquivo, salvar, pedir ao Node.js que execute e ler a saída no terminal.

Não utilizaremos HTML, interface, Git, npm, framework ou serviço externo.

Esta é a rota adotada para o percurso atual, mas ela ainda precisa ser validada com iniciantes em diferentes sistemas. Se a instalação impedir a atividade, há uma contingência ao final da preparação.

O ambiente é meio, não objetivo. Se a preparação dominar a sessão, use a contingência ao final do capítulo e registre a dificuldade para investigar depois.

Prepare o ambiente

Se você já possui Node.js e VS Code, vá para **Crie a pasta e o arquivo**.

Instale o Node.js

Acesse a [página oficial de download do Node.js](#) e escolha uma versão marcada como LTS. O número muda com o tempo; não procure uma versão exata citada em imagens ou outros materiais.

No Windows:

- 1 escolha o instalador correspondente ao computador — normalmente x64;
- 2 abra o arquivo terminado em .msi;
- 3 avance mantendo pasta e opções padrão;

- 4 deixe desmarcada qualquer opção de instalar ferramentas adicionais;
- 5 conclua a instalação.

No macOS, escolha o pacote compatível com o processador. No Linux, a instalação varia entre distribuições; use o pacote oficial correspondente ao seu sistema. Não instale ferramentas adicionais para este projeto.

Instale o editor

Acesse a [página oficial do Visual Studio Code](#).

No Windows, escolha **User Installer** para a arquitetura correspondente e execute o arquivo .exe. No macOS, escolha a versão **Universal** ou a correspondente ao processador, abra o arquivo .dmg e arraste Visual Studio Code.app para a pasta **Aplicativos**. No Linux, use o pacote da distribuição, como .deb ou .rpm, e siga o procedimento do gerenciador de pacotes do sistema.

Você não precisa criar conta, conectar GitHub, instalar extensões, ativar agentes de IA nem configurar outras linguagens.

Crie a pasta e o arquivo

No gerenciador de arquivos do sistema, entre em **Documentos** e crie a pasta:

LITERAL TÉCNICO

```
programacao-era-ia
```

No VS Code:

- 1 abra **Arquivo > Abrir Pasta** — ou **File > Open Folder**;
- 2 escolha programacao-era-ia;
- 3 confirme a abertura;
- 4 se aparecer uma pergunta sobre confiança nos autores, confirme: a pasta foi criada por você;
- 5 na lateral esquerda, clique no ícone de novo arquivo;
- 6 digite planejador.js e pressione Enter.

A terminação `.js` identifica o arquivo como JavaScript.

Na lateral esquerda, o nome `planejador.js` deve aparecer dentro de `programacao-era-ia`. Clique nele para confirmar que a área central mostra o arquivo vazio.

Se você abriu apenas um arquivo, e não a pasta, volte a **Arquivo > Abrir Pasta**. Manter a pasta aberta ajuda o terminal a começar no lugar correto e evita aprender navegação por diretórios antes desta atividade.

Criar ou salvar o arquivo não executa o programa. Salvar e executar serão ações separadas.

Abra o terminal e verifique o Node.js

No menu superior, escolha **Terminal > Novo Terminal** ou **Terminal > New Terminal**. O painel inferior é o terminal. Como a pasta foi aberta no editor, o terminal integrado normalmente começa nela.¹

No início da linha do terminal, algum texto indicará o local atual. Ele varia entre sistemas. Você não precisa interpretar tudo; procure apenas algo relacionado a `programacao-era-ia`.

Ao executar `node planejador.js`, o Node.js procurará esse nome na pasta atual. Se a mensagem disser que o arquivo não foi encontrado, confirme o nome na lateral e o local do terminal antes de modificar o código.

Digite:

TERMINAL

```
node --version
```

Pressione Enter. A saída deve começar com `v` e mostrar um número de versão. O número exato não importa.

Esse comando apenas confirma que o terminal encontra o Node.js. Se ele funciona e `node planejador.js` não, a dificuldade provavelmente está no arquivo, no nome ou na pasta atual.

Se aparecer uma mensagem dizendo que `node` não foi reconhecido:

¹ feche todas as janelas do VS Code;

- 2 abra o programa novamente;
- 3 reabra a pasta e crie outro terminal;
- 4 repita `node --version`.

Se continuar indisponível, registre a mensagem completa e use a contingência. Uma dificuldade de instalação não precisa encerrar o percurso.

Não instale gerenciadores, extensões ou pacotes no meio da atividade. Registre a mensagem para investigar depois sem ampliar o ambiente agora.

Primeira decisão — produzir uma saída

No arquivo `planejador.js`, escreva:

JAVASCRIPT

```
const minutos = 25
const travado = false

let proximoPasso = "Construa e teste uma melhoria."

console.log(proximoPasso)
```

As duas primeiras linhas representam as entradas. Usamos `const` porque elas não serão reatribuídas durante uma execução; quando quisermos outro caso, editaremos o arquivo e executaremos de novo.

`proximoPasso` guarda a mensagem. Usamos `let` porque ela mudará quando acrescentarmos as regras.

`console.log(...)` exibe no terminal o valor colocado entre os parênteses.

Aqui, `const` mantém as entradas fixas durante uma execução, enquanto `let` permite que a mensagem seja substituída pelas condições. Você não precisa memorizar todas as regras desses termos; precisa reconhecer qual valor muda neste caminho.

As entradas ainda não influenciam a saída. O primeiro objetivo é confirmar que o arquivo pode ser salvo, executado e observado.

Salve com `Ctrl + S` — ou `Command + S` no macOS. No terminal, execute:

TERMINAL

```
node planejador.js
```

A saída esperada é:

SAÍDA

```
Construa e teste uma melhoria.
```

Se aparecer a mesma frase com outra cor ou formatação do terminal, concentre-se no texto. Se não aparecer nada, confirme `console.log(proximoPasso)`. Se houver erro, leia a primeira linha antes de tentar outra mudança.

Salvar não é executar

Altere a mensagem no editor, mas não salve. Execute novamente. O terminal continuará usando a versão salva.

Agora salve e execute outra vez. A sequência é:

- 1 editar;
- 2 salvar;
- 3 executar;
- 4 observar.

Quando uma mudança parece não ter efeito, confirme primeiro se o arquivo foi salvo.

O terminal não atualiza sozinho: o Node.js lê a versão salva quando você envia `node planejador.js`.

Restaure a mensagem original antes de continuar.

Segunda decisão — priorizar o bloqueio

Se `travado` for `true`, a mensagem deve mudar. Insira este bloco antes de `console.log`:

JAVASCRIPT

```
if (travado) {  
  proximoPasso = "Explique o que você já tentou."  
}
```

if pode ser lido como “se”. Os parênteses contêm a condição; as chaves delimitam o que acontece quando ela é verdadeira. Copie a forma e concentre sua explicação no comportamento: se travado for verdadeiro, a mensagem muda.

Com travado igual a false, a atribuição não ocorre e a mensagem inicial permanece.

Antes de executar, faça uma previsão. Depois salve e rode o arquivo.

Altere somente a entrada para:

JAVASCRIPT

```
const travado = true
```

Preveja, salve e execute. A saída esperada é:

SAÍDA

```
Explique o que você já tentou.
```

Não avance até conseguir explicar por que a mensagem muda.

Se o arquivo parar de executar depois desse bloco, compare o trecho digitado com o exemplo. Procure aspas ausentes, parênteses incompletos e chaves sem par. Se executar, mas mantiver a mensagem inicial com travado = true, confirme o nome, o valor e se o arquivo foi salvo.

Terceira decisão — considerar os minutos

Resta tratar as sessões curtas. Substitua o bloco anterior por:

JAVASCRIPT

```
if (travado) {  
  proximoPasso = "Explique o que você já tentou."  
} else if (minutos < 20) {  
  proximoPasso = "Faça uma alteração pequena."  
}
```

else if pode ser lido como “caso contrário, se”. A ordem importa:

- 1 primeiro, o programa verifica travado;
- 2 somente se não estiver travado, verifica minutos < 20;
- 3 se nenhuma condição for verdadeira, mantém a mensagem inicial.

Agora existem três comportamentos. Não foi necessário um bloco final porque a mensagem longa já é o valor inicial; as condições a substituem quando há bloqueio ou pouco tempo.

Verifique antes de organizar

Altere somente as duas entradas e execute estes casos:

Minutos	travado	Resultado esperado
25	false	Construa e teste uma melhoria.
10	false	Faça uma alteração pequena.
25	true	Explique o que você já tentou.
20	false	Construa e teste uma melhoria.

Para cada linha: preveja, edite, salve, execute e compare.

Não altere a condição apenas para obrigar o programa a concordar com uma previsão errada. Volte à especificação. Se 20 minutos devem produzir a mensagem longa, minutos < 20 é falso nesse limite.

Se algum caso divergir, não continue. Observe os valores, a ordem, o símbolo < e o estado do arquivo salvo. Se a diferença não estiver clara, volte ao último incremento que funcionava e examine apenas o trecho acrescentado.

Por que criar uma função agora

A versão já cumpre os três comportamentos, mas a regra está misturada com as entradas e com a exibição.

Uma função reúne um comportamento sob um nome. Depois podemos fornecer valores diferentes e receber o resultado sem reescrever as condições. Ela aparece porque resolve essa necessidade visível.

Ela também separa dois momentos: declarar como a escolha funciona e chamar esse comportamento com entradas concretas. A declaração não produz uma recomendação sozinha. A chamada fornece os valores e recebe a mensagem.

Substitua o conteúdo do arquivo pela versão integral abaixo:

JAVASCRIPT

```
const minutos = 25
const travado = false

function escolherProximoPasso(minutos, travado) {
  if (travado) {
    return "Explique o que você já tentou."
  }

  if (minutos < 20) {
    return "Faça uma alteração pequena."
  }

  return "Construa e teste uma melhoria."
}

const proximoPasso = escolherProximoPasso(minutos, travado)

console.log(proximoPasso)
```

Observe as partes:

- `function escolherProximoPasso` declara e nomeia o comportamento;
- `(minutos, travado)` define os valores que a função receberá;
- as chaves delimitam seu corpo;
- `return` devolve uma mensagem e encerra aquela chamada;

- `escolherProximoPasso(minutos, travado)` chama a função;
- `proximoPasso` guarda o resultado;
- `console.log(proximoPasso)` o exibe.

Os nomes nos parênteses da declaração são **parâmetros**: representam as informações de que a função precisa. Na chamada, as variáveis externas fornecem os valores concretos. Acompanhe o caminho: valor definido no início → argumento enviado → parâmetro usado → mensagem devolvida.

Quando `travado` é verdadeiro, o primeiro `return` encerra a chamada. A condição de tempo não é alcançada. Se as duas condições forem falsas, a função chega ao retorno final.

O último `return` representa os casos restantes: o programa já descobriu que não há bloqueio e os minutos não estão abaixo de 20. Acompanhe mentalmente um caso de cada caminho. Se sua explicação com `travado = true` começar pelo tempo, volte à ordem da função.

Transformar a regra em função não deveria mudar o comportamento. Repita os quatro casos da tabela anterior para confirmar.

Se a versão anterior funcionava e a nova não, a reorganização introduziu uma divergência localizável. Compare primeiro a chamada, os parâmetros e os retornos; não volte a modificar a especificação.

Depois que a função executar, confirme pelo menos um caso de cada caminho e a fronteira de 20 minutos. A reorganização deveria preservar o comportamento.

Se uma etapa parar de funcionar

Antes de pedir que uma IA reescreva o arquivo:

- 1 identifique a última versão que executou corretamente;
- 2 observe somente o trecho acrescentado depois dela;
- 3 confirme que salvou;
- 4 leia a primeira mensagem de erro;
- 5 compare nomes usados em lugares diferentes;
- 6 verifique parênteses, aspas e chaves.

Se precisar de ajuda, envie o menor trecho relacionado e informe o esperado, o observado, a mensagem exata e a última alteração.

Compare isso com “não funciona”:

Altere o arquivo para usar a função. Esperava a mensagem curta com 10 minutos e travado = false, mas recebi a mensagem longa. Salvei o arquivo e não apareceu erro.

Agora existe um caminho de investigação. Uma resposta da IA só participa da correção quando uma mudança é aplicada, salva e executada.

USE IA PARA

- explicar uma linha específica;
- interpretar uma mensagem de erro;
- perguntar sobre `function`, `return` ou `console.log`;
- comparar sua leitura da ordem das decisões.

NÃO TERCEIRIZE

- a previsão de cada incremento;
- as mensagens e a prioridade do bloqueio;
- a decisão de avançar com uma etapa quebrada;
- a execução e a comparação dos casos.

VERIFIQUE

- no arquivo que você salvou;
- na saída do terminal ou playground;
- com combinações diferentes de entrada;
- confirmando que a ajuda não ampliou o escopo;
- explicando a mudança antes de aceitá-la.

Rota de contingência

Se Node.js ou VS Code impedir a continuidade, utilize temporariamente o [compilador JavaScript do Programiz](#).

- 1 abra a página;
- 2 remova o exemplo existente;
- 3 cole a versão integral do código;
- 4 selecione **Run**;
- 5 observe a área **Output**;
- 6 altere apenas as entradas;
- 7 execute novamente.

Não é necessário criar conta. Como interfaces online mudam, essa rota precisa ser verificada novamente antes da publicação final.

A contingência ocupa temporariamente o lugar do arquivo local, do Node.js e do terminal. Código e comportamento permanecem iguais. Registre a fricção se decidir retomar a rota local.

Se o playground acrescentar código automaticamente ou mostrar anúncios e recursos laterais, concentre-se no editor, no botão de execução e na saída. Não explore recursos de compartilhamento ou IA durante esta atividade.

A pequena vitória funcional

Deixe o arquivo com uma combinação válida, salve e execute:

TERMINAL

```
node planejador.js
```

A versão está funcionalmente concluída quando:

- o arquivo executa e exibe uma mensagem;
- o bloqueio possui prioridade;
- menos de 20 minutos produz a mensagem de alteração pequena;

- 20 minutos ou mais produz a mensagem de construir e testar;
- os quatro casos correspondem ao esperado.

O comportamento definido foi concluído. Explicar todas as partes, modificar a regra e recuperar uma falha pertence ao próximo capítulo.

Faça uma última execução com o arquivo salvo e mantenha a saída visível. Depois feche e abra novamente o VS Code, reabra a pasta e execute o comando. Repetir o caminho uma vez ajuda a separar o programa do estado momentâneo da interface.

Você não apenas criou uma pasta ou começou um projeto. Existe um programa que recebe valores, aplica decisões e produz uma recomendação verificável.

O que merece atenção ficou específico: salvar e executar; compreender `if`, função, parâmetros e `return` no contexto usado; construir por incrementos e comparar com a especificação. Validação, interação com usuário, testes automatizados e publicação continuam fora desta versão.

Antes de encerrar, aponte no arquivo as entradas, a função, os parâmetros, a condição prioritária, o limite, os retornos, a chamada e a saída.

Escolha um caso e narre o caminho: valores, condição, `return` e saída. Olhe o código se precisar; o objetivo é relacionar estrutura e comportamento. Se alguma parte só funcionou depois de copiar o trecho, registre-a. O Capítulo 8 transformará essa dependência em uma pergunta observável.

No próximo, a pergunta será mais exigente:

| Funcionou. Mas você entendeu?

Funcionou. Mas você entendeu?

O Planejador da Próxima Sessão já funciona. Você altera minutos e travado, executa o arquivo e recebe uma recomendação.

Isso é uma conquista real. A saída correta, porém, não mostra sozinha se você explica as decisões, altera a regra ou recupera o programa quando algo quebra. Fazer funcionar importa; agora vamos observar quanto controle você possui sobre o que construiu.

Comece por uma situação nova

Abra `planejador.js`, mas não execute ainda. Use:

JAVASCRIPT

```
const minutos = 10
const travado = true
```

As duas informações poderiam influenciar a resposta: há pouco tempo e existe um bloqueio.

Qual mensagem será exibida? Qual condição será avaliada primeiro? A condição dos minutos chegará a ser usada?

Registre sua previsão e só então execute.

A saída esperada é:

SAÍDA

```
Explique o que você já tentou.
```

Se sua previsão foi diferente, não apague a resposta. A diferença mostra qual parte do caminho precisa ser revista.

Esse caso acrescenta uma dificuldade: duas condições parecem pedir ações diferentes. A especificação precisa definir qual tem prioridade.

Depois da execução, teste `minutos = 10` com `travado = false`. A única mudança é o bloqueio. Se a mensagem também mudar, explique qual caminho passou a ser alcançado.

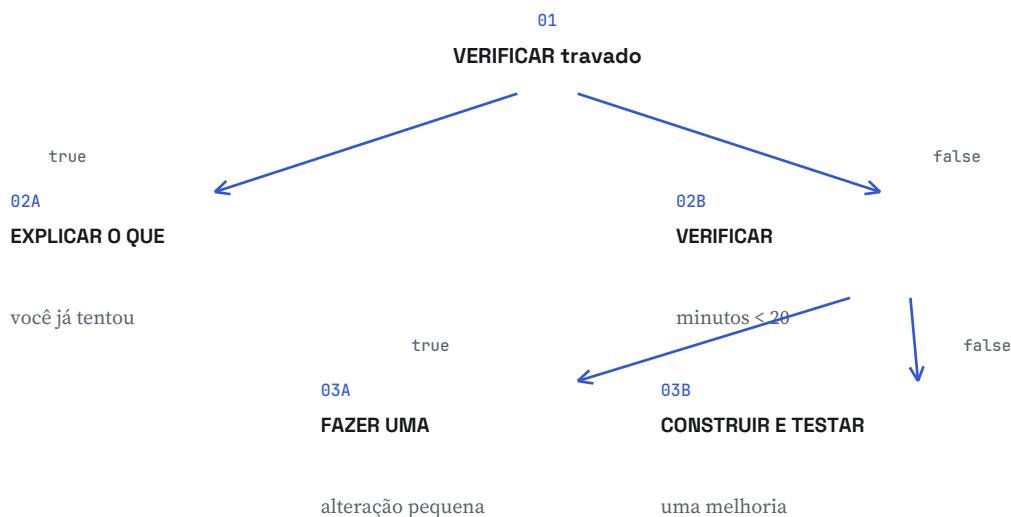
Explique a ordem das decisões

O bloqueio é verificado primeiro:

JAVASCRIPT

```
if (travado) {  
  return "Explique o que você já tentou."  
}
```

DIAGRAMA D5



Quando `travado` é `true`, `return` entrega a mensagem e encerra a chamada da função. A condição dos minutos não é alcançada.

Se `travado` é `false`, a execução continua:

JAVASCRIPT

```
if (minutos < 20) {  
  return "Faça uma alteração pequena."  
}
```

Se essa comparação também for falsa, resta:

JAVASCRIPT

```
return "Construa e teste uma melhoria."
```

Explique a ordem sem recitar o arquivo linha por linha. Uma versão possível é:

Primeiro o programa verifica o bloqueio. Se ele existe, devolve a primeira mensagem e termina. Se não existe, verifica o tempo. Com menos de 20 minutos, recomenda uma alteração pequena. Nos outros casos, recomenda construir e testar uma melhoria.

Agora responda consultando o código, se necessário:

- por que 10 minutos e `true` não produzem a mensagem de alteração pequena?
- por que 20 minutos e `false` produzem a recomendação mais longa?
- em quais pontos a função pode terminar?

Consultar o arquivo ou a documentação não invalida sua compreensão. O critério não é decorar símbolos; é reconstruir a decisão sabendo o que procurar.

Uma explicação útil deve sobreviver a pequenas mudanças. Se trocar 10 por 15, o caminho permanece? Se travado passar a `false`, qual condição decide? Formule a resposta com base na ordem.

Altere uma regra com intenção

O limite atual é 20:

JAVASCRIPT

```
if (minutos < 20) {
```

Vamos alterá-lo para 25:

JAVASCRIPT

```
if (minutos < 25) {
```

Antes de executar, defina:

Minutos	travado	Resultado esperado com a nova regra
24	false	Faça uma alteração pequena.
25	false	Construa e teste uma melhoria.
10	true	Explique o que você já tentou.

Salve e execute cada caso.

A troca de um número modificou uma decisão. Com `minutos < 25`, 24 entra no grupo das sessões curtas e 25 não entra. A prioridade do bloqueio permanece.

Depois, restaure o requisito original:

JAVASCRIPT

```
if (minutos < 20) {
```

O limite 20 não é correto por uma propriedade especial da linguagem. É correto porque pertence à especificação desta versão. Se o requisito muda, código e resultados esperados precisam mudar juntos.

Alterar intencionalmente também exige saber o que deve permanecer igual. A nova fronteira não deveria afetar a mensagem de bloqueio nem eliminar uma saída.

Se o teste com `travado = true` falhar depois da mudança, você terá evidência de que modificou mais do que o limite pretendido.

Quando o programa indica onde parou

Agora provoque uma falha controlada. Encontre:

JAVASCRIPT

```
const proximoPasso = escolherProximoPasso(minutos, travado)
```

Troque somente minutos por minuto:

JAVASCRIPT

```
const proximoPasso = escolherProximoPasso(minuto, travado)
```

Salve e execute:

TERMINAL

```
node planejador.js
```

Em vez da recomendação, aparecerá uma mensagem semelhante a:

ERRO

```
ReferenceError: minuto is not defined
```

O terminal poderá mostrar caminho, linha e outras informações. Comece pela parte direta:

ERRO

```
minuto is not defined
```

Naquele ponto, o JavaScript tentou usar o nome `minuto`, mas não encontrou uma variável definida com esse nome.

No início do arquivo, existe:

JAVASCRIPT

```
const minutos = 25
```


minuto e minutos são nomes diferentes. A função nem começa a decidir: antes da chamada, o JavaScript precisa encontrar os valores enviados como argumentos e falha ao procurar `minuto`.

Você não precisa memorizar todas as linhas do erro. Precisa extrair uma pista e relacioná-la ao código.

O caminho indica o arquivo; a linha aponta uma região; uma seta pode destacar parte da instrução. Comece pelo nome citado: procure `minuto` e compare com os nomes definidos.

Recupere e verifique

Antes de corrigir, formule a hipótese:

O programa parou porque a chamada usa `minuto`, mas a variável definida se chama `minutos`.

Corrija somente o nome, salve e execute novamente.

Se a recomendação reapareceu, sua hipótese ganhou apoio. Ainda assim, verifique os três comportamentos principais:

Minutos	travado	Resultado esperado
25	false	Construa e teste uma melhoria.
10	false	Faça uma alteração pequena.
25	true	Explique o que você já tentou.

Corrigir a interrupção não garante que as regras permanecem corretas. A recuperação termina quando o arquivo volta a executar e os comportamentos relevantes são verificados.

Se a correção não funcionar, volte à mensagem atual. Não continue investigando uma mensagem antiga se o programa agora apresenta outra pista. Cada execução atualiza o que sabemos sobre o estado do arquivo.

Renomear também parâmetros e condições poderia fazer o arquivo executar, mas mudaria partes sem necessidade. A correção mínima preserva mais informação sobre a causa.

O percurso foi:

- 1 observar a falha;
- 2 ler a pista principal;
- 3 localizar o nome indicado;
- 4 comparar com o código;
- 5 formular uma hipótese;
- 6 alterar uma coisa;
- 7 executar e verificar.

Isso não é um método para qualquer bug. É uma primeira prática de investigação.

Em programas maiores, uma falha pode atravessar arquivos, serviços e dados. Mesmo assim, a disciplina de registrar o esperado, localizar a primeira divergência observável e testar uma hipótese por vez continua útil.

Nem toda falha mostra uma mensagem

Troque temporariamente:

JAVASCRIPT

```
if (minutos < 20) {
```

por:

JAVASCRIPT

```
if (minutos > 20) {
```

O símbolo > significa “maior que”. Execute os três casos da tabela anterior.

O arquivo continua executando e nenhuma mensagem de erro aparece. Mesmo assim:

- com 10 minutos, recomenda construir e testar;
- com 25 minutos, recomenda uma alteração pequena;
- com bloqueio, a primeira mensagem ainda funciona.

Esse é um problema de lógica: a sintaxe é aceita, mas o comportamento contraria a especificação.

O caso de bloqueio ainda passa. Ele não compensa os outros resultados, mas ajuda a localizar a divergência na decisão de tempo.

Restaure < e teste novamente.

Ao final das experiências, deixe o arquivo na especificação original: limite 20, chamada com minutos no plural e três mensagens padronizadas. Isso evita começar o próximo ciclo a partir de uma alteração criada apenas para investigação.

“Não apareceu erro” é um critério fraco. A mensagem de erro ajuda quando a execução quebra. O comportamento esperado ajuda quando o programa executa, mas toma a decisão errada.

Por isso, depuração e teste se encontram. A mensagem oferece uma pista sobre a execução; os casos esperados oferecem pistas sobre a lógica.

Os dois tipos de falha pedem perguntas diferentes. No `ReferenceError`, qual nome não foi encontrado e onde a execução parou? No erro lógico, qual caso divergiu e qual regra poderia produzir a inversão?

IA durante a investigação

Você pode consultar uma IA quando encontrar uma mensagem desconhecida. A ordem da interação faz diferença.

USE IA PARA

pedir uma explicação do erro, comparar sua hipótese e sugerir casos adicionais.

NÃO TERCEIRIZE

a leitura inicial, sua primeira hipótese, a regra esperada e toda a verificação posterior.

VERIFIQUE

se a explicação menciona o nome presente no erro, se a correção modifica somente o necessário e se os três comportamentos continuam funcionando.

Uma pergunta útil seria:

Recebi `ReferenceError: minuto is not defined`. O que “not defined” indica e quais nomes devo comparar? Não reescreva o programa.

A IA pode reduzir o tempo da investigação sem eliminar sua hipótese e verificação. Se sugerir várias mudanças, peça que relacione cada uma à mensagem observada. Uma diferença singular/plural não justifica reorganizar o programa.

O que os testes demonstram — e o que não demonstram

Você previu, executou, alterou uma regra, provocou uma falha, interpretou uma mensagem e recuperou o programa. Isso fornece evidências de algum controle sobre o Planejador.

Não demonstra que:

- não existem outros defeitos;
- todos os valores possíveis foram verificados;
- entradas inadequadas são tratadas;
- o programa funcionará em qualquer contexto;
- você domina testes, JavaScript ou desenvolvimento de software.

Nesta versão, assumimos que `minutos` recebe um número válido, `travado` recebe um booleano, as entradas são editadas no arquivo e o resultado aparece no terminal. Não tratamos textos, valores negativos nem ausência de entrada. Também não criamos testes automatizados.

Esses assuntos existem, mas não são necessários para concluir o ciclo atual.

Um teste responde a uma pergunta específica. `20, false` verifica a fronteira; `10, true`, a prioridade; a nova execução depois do `ReferenceError`, a recuperação. Nenhum responde por todos os valores e contextos.

Escolher casos é parte da compreensão. Dez números aleatórios podem informar menos do que três casos escolhidos para separar abaixo, limite e acima — ou para colocar duas condições em conflito.

Registre também casos não executados, em vez de dizer “testei tudo” quando verificou apenas o núcleo.

Compreender não é recitar o arquivo

Você não precisa fechar o editor e reproduzir o programa inteiro de memória. Memorizar parte da sintaxe pode acontecer com a prática, mas não é o melhor critério agora.

Consulte o arquivo e explique o objetivo, as entradas, a prioridade do bloqueio, a fronteira de 20 minutos e o papel de `return`. Depois, mostre onde o resultado é exibido e por que `minuto` interrompeu a execução.

Se uma resposta estiver incompleta, você encontrou um ponto de estudo. Não precisa escondê-lo nem transformar a dúvida em motivo para abandonar o projeto.

Registre a fronteira com duas colunas:

Consigo explicar ou fazer	Ainda preciso investigar
Como travado recebe prioridade	Todas as informações adicionais exibidas pelo Node.js no erro
Como o limite altera a mensagem	Como validar uma entrada inadequada
Como corrigir <code>minuto</code> para <code>minutos</code>	Como criar testes automatizados

Os exemplos não são respostas obrigatórias. O que estiver na segunda coluna pode orientar um ciclo futuro sem entrar nesta versão.

Sua demonstração de controle

Sem adicionar funcionalidades, escolha três combinações de entrada, preveja as mensagens e explique o caminho pelas condições. Execute e compare. Depois, altere o limite, defina dois resultados, teste e restaure a regra original. Por fim, explique o `ReferenceError` e registre uma parte que ainda não compreende completamente.

Uma declaração honesta pode ser:

Entendo como as entradas percorrem as condições e produzem uma das mensagens, mas ainda não compreendo todas as linhas exibidas no erro.

Isso é melhor do que fingir domínio. Prever, explicar, alterar e testar fornecem evidências de controle sobre este programa; não provam competência geral em programação.

Consultar o capítulo não transforma a atividade em fracasso. Se você só consegue responder na ordem dos exemplos, crie um caso próprio, narre o caminho e execute.

A vitória é específica: você acompanhou decisões, modificou uma regra, encontrou uma falha, interpretou uma pista, recuperou o código e verificou o resultado.

Você ainda não demonstrou domínio geral em programação, mas possui ações concretas para distinguir funcionamento, compreensão parcial e perda de controle.

O próximo passo não é aprender tudo. É escolher qual novo ciclo merece sua atenção.



Escolha o próximo ciclo, não a vida inteira

Você chegou até aqui com algo que não tinha no início: uma experiência completa, ainda que pequena.

Transformou um problema em comportamento, escreveu regras, executou um programa, previu resultados, alterou decisões, investigou uma falha e verificou o que aconteceu. Também reconheceu os limites do que compreende.

Agora surge outro risco: voltar ao excesso de informação e acreditar que precisa escolher hoje toda a trajetória profissional dos próximos anos.

Não precisa.

Sua próxima decisão é sobre o próximo ciclo.

Uma direção não é uma sentença

Escolher uma direção significa concentrar atenção por tempo suficiente para construir algo e aprender com o processo. Você pode experimentar interfaces e descobrir que prefere dados, começar com jogos e depois gostar de ferramentas. O problema não é mudar; é mudar antes de concluir uma experiência que forneça informação sobre a escolha.

Uma direção precisa durar tempo suficiente para você encontrar dificuldades reais, resolver algumas delas e observar se ainda existe interesse. Isso é diferente de assumir um compromisso permanente com uma linguagem ou cargo.

Defina o que faria você reconsiderar: falta de interesse depois de uma experiência concluída, dependências que não cabem nos recursos atuais ou descoberta de um resultado mais significativo. Mudar com base em experiência é diferente de reagir a cada tendência.

Os fundamentos praticados neste livro não pertencem exclusivamente ao JavaScript: representar valores, organizar decisões, dividir problemas, definir comportamentos, observar, investigar e testar acompanham você mesmo quando sintaxe e ambiente mudam.

Por isso, comece pelo tipo de resultado que gostaria de produzir. A tecnologia entra como parte da resposta.

Minha trajetória também não começou com um plano completo

Eu não comecei aos 13 anos com uma carreira definida. Queria criar meu servidor de Tibia para jogar com amigos. Isso me levou a mapas, quests, monstros, NPCs, magias, XML e Lua.

Quando o servidor passou a receber outros jogadores, apareceu outra necessidade: criar contas e personagens. Cheguei ao site em PHP, Apache, MySQL e XAMPP. Eu conseguia instalar, configurar, ler partes do código e alterar a aparência, mas ainda não compreendia a lógica suficiente para criar funcionalidades novas.

Depois, estudei HTML e CSS por conta própria. Quando entrei em um curso de informática e me tornei estagiário, cheguei a ajudar alunos com dúvidas de HTML antes de cursar aquele módulo. Anos mais tarde, front-end se tornou profissão.

Não conto isso como trajetória perfeita nem como caminho a ser copiado. Curiosidade, oportunidades, limitações e problemas concretos foram influenciando cada passo.

A lição não é que todo iniciante deveria começar por front-end ou por um servidor de jogo. É que uma direção pode nascer de algo que você quer modificar, entender ou construir.

Até hoje, criar um servidor old-school continua sendo um sonho meu. O sonho aponta uma direção; o próximo ciclo define o que cabe agora.

“Escolher uma área” nem sempre acontece antes da prática. Muitas vezes, ela recebe um nome depois que problemas, ferramentas e oportunidades começam a se repetir. Se eu pudesse organizar melhor aquela exploração, manteria a curiosidade e acrescentaria critérios para

perceber onde eu apenas configurava e onde já conseguia criar e depurar uma decisão.

Comece pelo resultado

Em vez de perguntar primeiro “qual área devo escolher?”, experimente:

- o que eu gostaria de ver funcionando?
- quero criar algo com que uma pessoa possa interagir?
- quero reduzir uma tarefa repetitiva?
- quero organizar informações?
- quero simular regras e consequências?
- quero entender como partes de um sistema se conectam?

Algumas direções possíveis:

Interesse	Resultado pequeno	Pergunta para o ciclo
Experiência interativa	Uma página que reage a uma ação	Como o programa responde ao que uma pessoa faz?
Automação	Um programa que aplica regras a uma tarefa repetitiva	Quais passos podem ser transformados em código?
Dados	Um programa que resume uma pequena coleção	Como representar valores e extrair algo útil?
Aplicações	Uma versão mínima que recebe e preserva uma informação	Como as partes trocam dados e mantêm estado?
Jogos ou simulações	Uma experiência curta com escolhas e consequências	Como representar regras que mudam o resultado?

Esses resultados não têm a mesma dificuldade. Preservar dados costuma exigir mais camadas do que reagir a uma ação simples; uma automação com serviço externo é diferente de uma regra local.

Depois de escolher o tipo de resultado, aplique o mesmo critério do Capítulo 6: qual é a menor versão que ainda produz algo observável?

Se duas direções parecem igualmente interessantes, não tente resolvê-las apenas comparando opiniões. Escolha a que permite um experimento

menor com os recursos atuais. Um ciclo concluído fornecerá informação que nenhuma lista genérica consegue antecipar.

Transforme interesse em pergunta

“Quero aprender front-end” ou “quero aprender automação” ainda é amplo.

Transforme a direção em algo que possa ser investigado:

Como faço uma página responder a uma ação?

Como transformo uma regra repetitiva em um programa executável?

Como represento uma coleção de valores e calculo um resumo?

Como crio uma escolha que altera o resultado de uma simulação?

Compare “quero dominar desenvolvimento web” com:

Quero criar uma página simples na qual uma pessoa informa o tempo disponível e recebe uma recomendação.

A segunda frase ajuda a identificar o comportamento, os conceitos necessários, as exclusões e a forma de verificar.

Uma pergunta de aprendizagem deve trazer algo novo sem acumular tantas dependências que o comportamento desapareça. “Como recebo uma ação e mostro uma resposta?” delimita melhor o estudo do que “como construo uma plataforma completa?”.

Se a pergunta exigir cinco conceitos desconhecidos antes de qualquer execução, reduza novamente. Talvez o ciclo atual seja apenas compreender um deles em um experimento menor.

Você não escolheu sua vida inteira. Escolheu uma pergunta que consegue investigar construindo.

Continuar com JavaScript ou mudar?

JavaScript foi a linguagem de laboratório deste livro. Ele não é a melhor escolha universal.

Continuar por mais um ciclo pode liberar atenção para o problema, porque você não precisa reaprender imediatamente cada símbolo. Outra linguagem pode fazer sentido quando houver um motivo ligado ao projeto ou ambiente.

A pergunta útil é:

Qual linguagem me permite investigar este problema sem acrescentar complexidade desnecessária?

Se mudar de linguagem, preserve o tamanho do projeto. Não troque de ambiente e, ao mesmo tempo, tente construir um sistema enorme.

Permanecer com a mesma linguagem por algum tempo reduz mudanças simultâneas. Se mudar, mantenha problema e escopo pequenos para distinguir as dificuldades do novo ambiente das regras do projeto.

Reorganize as prioridades uma vez

As quatro categorias não formam uma lista definitiva. Elas mudam com o objetivo:

Categoria	Pergunta para decidir
APRENDA AGORA	Sem compreender isto, consigo construir o comportamento principal?
PRATIQUE AGORA	Esta capacidade se desenvolve principalmente durante a construção?
ENTENDA QUE EXISTE	Preciso reconhecer esta parte sem aprofundá-la neste ciclo?
ESTUDE DEPOIS	O assunto é interessante, mas não participa do resultado atual?

Uma tecnologia que ficou para depois neste livro pode subir de prioridade no próximo projeto. O que não deve acontecer é tudo subir ao mesmo tempo.

Considere uma página interativa como próximo ciclo. A estrutura mínima da página e a conexão com JavaScript podem entrar em **APRENDA AGORA**. Reaproveitar a função, testar as três mensagens e

investigar erros entram em **PRATIQUE AGORA**. Publicação e acessibilidade precisam ser reconhecidas, mas talvez ainda estejam em **ENTENDA QUE EXISTE**. Framework e autenticação continuam em **ESTUDE DEPOIS**.

Essa é apenas uma aplicação das categorias, não uma continuação obrigatória. Se sua direção for dados, automação ou jogos, as prioridades serão outras.

O valor do sistema está justamente em mudar com o resultado escolhido.

Um exemplo compacto

Imagine que você queira reduzir uma tarefa repetitiva. A direção não precisa ser “virar especialista em automação”. Pode ser:

Quero criar um programa que recebe três valores de uma tarefa e recomenda uma prioridade.

Os conhecimentos necessários para representar os valores e escrever as condições entram em **APRENDA AGORA**. Formular regras, prever casos e investigar divergências entram em **PRATIQUE AGORA**. Você pode reconhecer que existem interfaces, armazenamento e publicação sem aprofundá-los. Integrações com serviços e infraestrutura permanecem para depois.

O projeto termina quando as combinações definidas produzem as prioridades esperadas e você consegue alterar uma regra sem perder os casos anteriores.

O ciclo escolheu uma experiência, não uma carreira.

Se seu objetivo inclui o primeiro emprego

Conseguir o primeiro emprego pode orientar o próximo ciclo, mas não serve como critério de conclusão: a contratação não depende apenas de você. O que você controla é concluir um projeto, explicar o problema e suas decisões, registrar como verificou, reconhecer limites e apresentar o resultado com honestidade.

Portfólio não é apenas uma coleção de telas. Um projeto pequeno e compreendido pode mostrar mais sobre seu trabalho do que várias camadas que você não consegue explicar. Isso não torna um programa

no terminal suficiente para qualquer vaga; indica uma progressão possível: concluir o núcleo e, quando o objetivo justificar, melhorar documentação, apresentação e publicação.

Você pode estudar e se candidatar ao mesmo tempo, conforme os requisitos concretos das oportunidades. Não existe promessa de contratação, mas o objetivo externo pode ser convertido em ações que você consegue executar e avaliar.

IA amplia opções; não escolhe por você

Uma IA pode relacionar objetivos a conceitos, comparar a complexidade de ideias e ajudar a reduzir projetos. Mas ela não conhece automaticamente seu tempo, recursos, dificuldades ou curiosidade.

Em vez de perguntar “qual é a melhor área para mim?”, experimente:

Quero experimentar uma ferramenta que reduza uma tarefa repetitiva. Sugira três projetos sem banco de dados, contas ou serviços externos. Para cada um, indique o comportamento mínimo e os conceitos novos. Não escolha por mim.

Use a resposta para ampliar opções. Não terceirize o motivo, o resultado desejado, os limites nem a avaliação sobre continuar.

Uma sugestão detalhada pode parecer segura e ainda presumir tempo, dinheiro ou conhecimentos que você não possui. Antes de adotá-la, conte os conceitos novos, remova tecnologia desnecessária e confirme se o primeiro comportamento pode ser testado.

A IA também pode comparar o planejado com o construído e organizar dúvidas. A decisão de aprofundar, mudar ou parar continua sendo sua.

Defina quando o ciclo termina

Sem critério de conclusão, uma página simples ganha cadastro, banco, painel, aplicativo e integração com IA. O projeto volta a representar tudo que você ainda não sabe.

Defina uma evidência limitada:

O ciclo estará concluído quando outra pessoa puder informar dois valores e receber uma das três recomendações previstas.

Ou:

O ciclo estará concluído quando a simulação aceitar duas escolhas e produzir consequências diferentes conforme as regras definidas.

O critério não prova domínio. Impede que o objetivo se mova continuamente.

Evite critérios como “quando eu souber bastante” ou “quando estiver profissional”. Eles não descrevem comportamento observável e podem manter o ciclo aberto indefinidamente. Prefira algo que outra pessoa consiga ver ou que você consiga executar e comparar.

Também escolha uma rotina possível. “Todos os dias por quatro horas” não ajuda se essa disponibilidade não existe. Uma rotina menor e repetível produz mais oportunidades de fechar ciclos do que um plano ideal que sempre precisa ser adiado.

Defina também o que fará numa sessão curta ou diante de um bloqueio. Depois da conclusão, revise o que passou a fazer, o que ainda depende de exemplos e se deseja aprofundar ou experimentar outra direção. Assim, a mudança nasce da experiência, não apenas da primeira dificuldade.

Escreva seu próximo ciclo

Transforme sua direção em uma decisão concreta:

1. Direção e pergunta

Que resultado quero construir, por que merece atenção agora e o que quero compreender?

2. Projeto pequeno

O que estará funcionando ao final? Qual é o comportamento principal?

3. Limites e prioridades

O que ficará fora? Classifique somente o necessário em **APRENDA AGORA, PRATIQUE AGORA, ENTENDA QUE EXISTE** e **ESTUDE DEPOIS**.

4. Evidências e conclusão

O que conseguirei prever, explicar, alterar e testar? Qual resultado indicará que esta versão terminou?

5. Primeira ação e revisão

Qual ação concreta começa o ciclo e quando avaliarei se continuo, aprofundo ou mudo?

Evite preencher a primeira ação com “estudar mais”. Prefira criar o arquivo, executar um exemplo, escrever comportamentos esperados, escolher dados ou reduzir a ideia a uma função.

Antes de encerrar, leia o plano inteiro e procure contradições. O projeto pequeno realmente responde à pergunta de aprendizagem? Os itens em **APRENDA AGORA** são necessários para o comportamento? O critério de conclusão pode ser observado? A primeira ação aproxima você do resultado ou apenas abre outro período de preparação?

Se o plano ainda exige decidir carreira, linguagem, ferramentas e arquitetura antes da primeira execução, reduza-o. Seu próximo ciclo deve produzir experiência para melhorar decisões futuras, não exigir que todas elas estejam resolvidas agora.

Guarde o plano onde consiga revisá-lo. Ele não é contrato permanente; é uma referência para perceber quando uma nova ideia ajuda o objetivo ou desvia atenção. Um ciclo futuro pode recuperar algo que ficou para depois sem transformar todas as possibilidades em urgência.

Você não terminou de aprender. Terminou este ciclo.

No início deste livro, o problema não era falta de informação.

Era excesso de informação sem direção.

Você via linguagens, ferramentas, frameworks, cursos, agentes e opiniões contraditórias. Não sabia o que precisava aprender, em que ordem ou quanto deveria entregar à IA.

Este livro não removeu todas essas possibilidades. Também não entregou um caminho capaz de prever toda a sua trajetória.

O que ele fez foi oferecer uma forma de avançar apesar da incerteza.

Você escolheu o necessário, deixou assuntos para depois, transformou um problema em comportamento e construiu uma versão pequena.

Depois, foi além do resultado: previu, explicou, alterou, testou, encontrou uma falha e recuperou o controle.

Isso não significa que você domina programação.

Significa que já não precisa tratar cada novo termo como uma ordem para mudar de caminho.

Na era da IA, gerar código ficou mais acessível. Explicações, exemplos e alternativas também podem chegar mais rápido. Mas rapidez e conclusão funcional continuam sem demonstrar, por si mesmas, que houve aprendizagem.

Seu papel não é rejeitar essas ferramentas nem entregar todo o raciocínio a elas.

É continuar fechando ciclos:

PREVER → EXECUTAR → OBSERVAR → EXPLICAR → ALTERAR → TESTAR

À medida que os projetos crescerem, novas ferramentas e camadas aparecerão. Algumas entrarão em **APRENDA AGORA**. Outras permanecerão em **ENTENDA QUE EXISTE** ou **ESTUDE DEPOIS**.

Você não precisa carregar todas ao mesmo tempo.

Escolha um problema. Reduza até caber. Construa. Verifique. Reconheça o que ainda não entende. Então escolha novamente.

Não decida sua vida inteira hoje.

Decida o próximo ciclo — e comece pela primeira ação.

Referências

Estudos e publicações

CHI, Michelene T. H.; BASSOK, Miriam; LEWIS, Matthew W.; REIMANN, Peter; GLASER, Robert. *Self-Explanations: How Students Study and Use Examples in Learning to Solve Problems*. Cognitive Science, v. 13, n. 2, p. 145–182, 1989. Disponível em: https://doi.org/10.1207/s15516709cog1302_1.

KAZEMITABAAR, Majeed; CHOW, Justin; MA, Carl Ka To; ERICSON, Barbara J.; WEINTROP, David; GROSSMAN, Tovi. *Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming*. Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, artigo 455, p. 1–23, 2023. Disponível em: <https://doi.org/10.1145/3544548.3580919>.

SENTANCE, Sue; WAITE, Jane; KALLIA, Maria. *Teaching computer programming with PRIMM: a sociocultural perspective*. Computer Science Education, v. 29, n. 2–3, p. 136–176, 2019. Disponível em: <https://doi.org/10.1080/08993408.2019.1608781>.

SHEN, Judy Hanwen; TAMKIN, Alex. *How AI Impacts Skill Formation*. arXiv:2601.20245v2, preprint, 2026. Disponível em: <https://arxiv.org/abs/2601.20245>.

Documentação e recursos técnicos

GOOGLE. *Run JavaScript in the Console*. Chrome for Developers. Disponível em: <https://developer.chrome.com/docs/devtools/console/javascript>. Acesso em: 23 ago. 2026.

GITHUB. *Application card: GitHub Copilot Chat*. Documentação do GitHub Copilot. Disponível em: <https://docs.github.com/en/copilot/responsible-use/chat>. Acesso em: 23 ago. 2026.

MICROSOFT. *Keyboard shortcuts — Microsoft Edge DevTools*. Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/microsoft-edge/devtools/shortcuts/>. Acesso em: 23 ago. 2026.

MICROSOFT. *Download Visual Studio Code*. Disponível em: <https://code.visualstudio.com/download>. Acesso em: 23 ago. 2026.

MICROSOFT. *Getting started with the terminal*. Visual Studio Code Documentation. Disponível em: <https://code.visualstudio.com/docs/terminal/getting-started>. Acesso em: 23 ago. 2026.

NODE.JS. *Download Node.js*. Disponível em: <https://nodejs.org/en/download>. Acesso em: 23 ago. 2026.

PROGRAMIZ. *JavaScript Online Compiler*. Disponível em: <https://www.programiz.com/javascript/online-compiler/>. Acesso em: 23 ago. 2026.

Sobre o autor

Marllon Panisset é Desenvolvedor Front-End e estudante de Engenharia de Software. Sua relação com tecnologia começou aos 13 anos, quando a vontade de criar um servidor de Tibia para jogar com amigos o levou aos OTServers e aos primeiros contatos com XML, Lua, PHP e HTML.

Antes de compreender programação de forma estruturada, aprendeu a instalar, configurar, explorar arquivos, modificar comportamentos e testar resultados. Essa experiência permanece como parte de sua maneira de pensar aprendizagem: curiosidade e experimentação abrem caminhos, mas fazer algo funcionar não significa compreender todas as decisões envolvidas.

Hoje, seus interesses incluem programação, inteligência artificial, engenharia de software, ferramentas para desenvolvedores e construção de projetos. No MarllonCode, compartilha estudos, experiências e investigações sobre como aprender e desenvolver software em uma época na qual gerar código ficou mais acessível — e compreender o que foi construído continua importante.

Sobre o MarllonCode

MarllonCode é um projeto de conteúdo sobre programação, inteligência artificial, engenharia de software, ferramentas para desenvolvedores, construção de projetos e aprendizagem em público.

A proposta é explorar tecnologia com clareza e senso crítico, transformando excesso de informação em decisões, experimentos e próximos passos que possam ser compreendidos e verificados.

Continue o próximo ciclo

Se este livro ajudou você a escolher o próximo passo, o percurso pode continuar no MarllonCode.

Lá, você poderá acompanhar novos estudos, experimentos, projetos, materiais e atualizações de futuras edições — sem precisar esperar até se sentir completamente pronto para começar.

Escolha seu próximo ciclo, execute a primeira ação e compartilhe o que conseguiu construir e compreender.

Instagram: @marlloncode